Steady Temperature Distribution in a Two-Dimensional Channel Flow

A Finite-Volume Study of Convection, Thermal Diffusion, and Viscous Dissipation

Arian Shahnoori

October 30, 2026

Abstract

A numerical solver is developed to investigate the steady-state temperature distribution in a two-dimensional channel flow. The governing energy equation accounts for convective transport, thermal diffusion, and viscous dissipation. It is discretized in space using a second-order, cell-centered finite-volume formulation on a structured Cartesian grid. The resulting semi-discrete system is advanced in pseudo-time using four numerical methods: an approximate factorization method, a sparse direct implicit method, and second- and fourth-order explicit Runge–Kutta methods.

The numerical study examines the stability and convergence characteristics of the time-integration methods, including the influence of time-step selection on computational performance. Solver performance is compared in terms of convergence history, iteration count, and computation time. Finally, parameter studies investigate the effects of the Peclet number, the Brinkman number, and the convection coefficient on the resulting temperature field. Temperature contours and wall-normal profiles are used to examine the relative contributions of convection, thermal diffusion, and viscous dissipation.

Physical Problem	Two-Dimensional Channel-Flow Heat Transfer
Domain	Rectangular Channel ($L_x = 5$, $L_y = 1$)
Velocity Field	Prescribed Fully Developed Laminar Profile
Governing Equation	Convection–Diffusion Energy Equation with Viscous Dissipation
Spatial Discretization	Cell-Centered Finite Volume Method
Spatial Accuracy	Second Order
Time Integration	Pseudo-Time Marching to Steady State
Solution Methods	APF, Sparse Direct Implicit, RK2, RK4
Stability Analysis	Eigenvalue Analysis and Amplification Factors
Parameter Study	Pe, Br, and Convection Coefficient c
Programming Language	Python
Performance Evaluation	Residual Convergence, Iterations, and Computation Time

Contents

1 Problem Description **5**

1.1 Physical Domain and Flow Conditions 5

1.2 Boundary and Initial Conditions 5

2 Governing Equation and Spatial Discretization **7**

2.1 Dimensionless Energy Equation 7

2.2 Finite-Volume Formulation 7

2.3 Spatial Discretization of the Advective Terms 8

2.4 Spatial Discretization of the Diffusive Term 9

2.5 Spatial Discretization of the Viscous-Dissipation Source 9

2.6 Complete Spatially Discretized Equation 10

2.7 Implementation of Boundary Conditions Using Ghost Cells 11

2.7.1 Dirichlet Boundary Conditions 11

2.7.2 Outlet Boundary Condition 12

2.7.3 Boundary Conditions in Delta Form 12

3 Temporal Discretization **14**

3.1 Approximate Factorization 14

3.1.1 Implicit Euler Discretization 14

3.1.2 Delta Formulation 14

3.1.3 Approximate Factorization 15

3.1.4 Alternating-Direction Implicit Solution 16

3.2 Explicit Runge–Kutta Methods 17

3.2.1 Second-Order Runge–Kutta Method 17

3.2.2 Fourth-Order Runge–Kutta Method 18

3.3 Direct Sparse Solution 19

3.3.1 Linearization of the Spatial Operator 19

3.3.2 Sparse Matrix Formulation 19

3.3.3 Implicit Euler Time Integration 20

3.4 Convergence Assessment 21

4 Temporal Accuracy and Stability Analysis **22**

4.1 Semi-Discrete System and Eigenvalue Analysis . . . 22
4.2 Amplification Factors and Temporal Accuracy . . . 22
4.2.1 Implicit Euler . . . 23
4.2.2 Second-Order Runge–Kutta Method . . . 24
4.2.3 Fourth-Order Runge–Kutta Method . . . 24
4.3 Numerical Assessment of Stability . . . 25

5 Numerical Accuracy Assessment 27
5.1 Accuracy Assessment Using an Exact Solution . . . 27
5.2 Grid Convergence Without an Exact Solution . . . 27

6 Computational Implementation 29
6.1 Grid Generation . . . 29
6.2 Initialization of the Solution Fields . . . 29
6.3 Boundary-Condition Implementation . . . 29
6.4 Convergence Monitoring and Diagnostics . . . 30
6.4.1 Normalized L_p Error . . . 30
6.4.2 Steady-State Residual . . . 31
6.5 Approximate Factorization Method . . . 32
6.5.1 Approximate Factorization Formulation . . . 32
6.5.2 Construction of the Spatial Residual . . . 34
6.5.3 Boundary Conditions in the Factorized Systems . . . 34
6.5.4 Streamwise System . . . 35
6.5.5 Wall-Normal System . . . 36
6.5.6 APF Iteration and Steady-State Convergence . . . 37
6.6 Explicit Runge–Kutta Implementation . . . 39
6.6.1 Stage Evaluation and Temperature Update . . . 39
6.6.2 Pseudo-Time Marching and Convergence . . . 40
6.7 Direct Sparse Solver Implementation . . . 41
6.7.1 Sparse Spatial Matrix Assembly . . . 42
6.7.2 Construction of the Forcing Vector . . . 44
6.7.3 Implicit Euler Integration and Sparse LU Factorization . . . 45
6.7.4 Verification of the Direct Solver . . . 47

7 Results **48**

7.1 Grid Convergence Without an Exact Solution 48

7.1.1 Successive-Grid Differences 48

7.1.2 Observed Spatial Convergence Order 49

7.1.3 Grid-Convergence Results for All Solvers 50

7.2 Temporal Accuracy Assessment 51

7.2.1 Successive-Time-Step Differences 52

7.2.2 Observed Temporal Convergence Order 52

7.2.3 Temporal Accuracy Results 53

7.3 Stability and Time-Step Efficiency 55

7.3.1 Stability Analysis 55

7.3.2 Time-Step Efficiency 57

7.3.3 Comparison of the Converged Solutions 58

7.4 Comparison of Solver Performance 58

7.5 Pseudo-Time Evolution of the Temperature Field 60

7.6 Parameter Study 61

7.6.1 Effect of the Brinkman Number 61

7.6.2 Effect of the Convection Coefficient 62

7.6.3 Effect of the Peclet Number 63

7.6.4 Temperature Cross-Sections 65

8 Conclusion **67**

1. Problem Description

1.1 Physical Domain and Flow Conditions

The objective of this project is to obtain a numerical solution to the two-dimensional energy equation for laminar, incompressible flow through a rectangular channel. The computational domain has a length of $L_x = 5$ and a height of $L_y = 1$. A uniform, cell-centered grid is used to discretize the domain.

The flow is assumed to be fully developed, with a prescribed velocity profile in the streamwise direction and no velocity component in the transverse direction. The dimensionless velocity field is defined as

$$u(y) = 6\overline{u}\, y(1-y), \qquad v(y) = 0, \tag{1}$$

where $\overline{u}$ is the mean streamwise velocity. For $\overline{u} = 3$, the velocity profile becomes

$$\boxed{u(y) = 18y(1-y), \qquad v(y) = 0.} \tag{2}$$

The nominal flow parameters are

$$\mathrm{Re} = 25, \qquad \mathrm{Pr} = 0.7, \qquad \mathrm{Ec} = 0.1, \tag{3}$$

where Re, Pr, and Ec denote the Reynolds, Prandtl, and Eckert numbers, respectively.

1.2 Boundary and Initial Conditions

The temperature field is subject to prescribed thermal conditions at the channel boundaries. The lower and upper walls are maintained at constant dimensionless temperatures of zero and one, respectively:

$$T(x, 0, t) = 0, \tag{4}$$
$$T(x, 1, t) = 1. \tag{5}$$

At the inlet, the temperature is prescribed using the linear profile

$$T(0, y, t) = T_{\mathrm{in}}(y) = y. \tag{6}$$

At the outlet, a zero streamwise temperature gradient is applied:

$$\left.\frac{\partial T}{\partial x}\right|_{x=L_x} = 0. \tag{7}$$

This condition represents zero conductive heat flux in the streamwise direction at the outlet.

The initial temperature distribution throughout the domain is specified as

$$\boxed{T(x, y, 0) = y.} \tag{8}$$

Thus, the initial temperature profile matches the prescribed inlet profile and varies linearly

between the lower and upper wall temperatures. Starting from this initial condition, the temperature field evolves under the combined effects of advection, thermal diffusion, and viscous dissipation until a steady-state solution is obtained.

2. Governing Equation and Spatial Discretization

2.1 Dimensionless Energy Equation

The objective of this project is to develop a numerical solution for the two-dimensional energy equation of an incompressible flow. The equation describes the evolution of temperature under the combined effects of advection, diffusion, and viscous dissipation. Its dimensionless form is given by

$$\begin{aligned}\frac{\partial T}{\partial t}+\frac{\partial (uT)}{\partial x}+\frac{\partial (vT)}{\partial y} &= \frac{1}{\mathrm{Re}\,\mathrm{Pr}}\nabla^2 T \\ &+\frac{\mathrm{Ec}}{\mathrm{Re}}\left[2\left(\frac{\partial u}{\partial x}\right)^2+2\left(\frac{\partial v}{\partial y}\right)^2+\left(\frac{\partial u}{\partial y}+\frac{\partial v}{\partial x}\right)^2\right],\end{aligned} \tag{9}$$

where T is the dimensionless temperature, u and v are the velocity components in the x- and y-directions, respectively. The dimensionless parameters Re, Pr, and Ec denote the Reynolds, Prandtl, and Eckert numbers.

The left-hand side of Equation (9) contains the temporal and advective contributions. The first term on the right-hand side represents thermal diffusion, while the second term accounts for viscous dissipation, which represents the conversion of mechanical energy into thermal energy due to velocity gradients. This contribution is treated as a source term, denoted by S.

2.2 Finite-Volume Formulation

The governing equation is discretized using the finite-volume method (FVM). The computational domain is divided into rectangular control volumes, with the solution variables stored at their centers. Integrating Equation (9) over a control volume and applying the divergence theorem to the advective and diffusive terms converts the corresponding volume integrals into surface integrals over the control-volume faces.

For a two-dimensional rectangular control volume, the surface integrals are evaluated as contributions from the east, west, north, and south faces. The resulting semi-discrete equation is

$$\begin{aligned}\frac{d\overline{T}_{ij}}{dt} = &-\frac{F^a_e-F^a_w}{\Delta x}-\frac{F^a_n-F^a_s}{\Delta y} \\ &+\frac{1}{\mathrm{Re}\,\mathrm{Pr}}\left[\frac{F^d_e-F^d_w}{\Delta x}+\frac{F^d_n-F^d_s}{\Delta y}\right]+\overline{S}_{ij},\end{aligned} \tag{10}$$

where $\overline{T}_{ij}$ and $\overline{S}_{ij}$ are the control-volume averages of temperature and the source term. The quantities F^a and F^d denote the normal advective flux and the normal temperature gradient, respectively. The subscripts e, w, n, and s represent the east, west, north, and south faces. The control-volume dimensions are Δx and Δy.

The surface integrals are approximated using values at the face centers. Since the numerical solution is stored at cell centers, the face values are reconstructed from the available cell-centered quantities. Second-order central approximations are employed for the spatial discretization.

2.3 Spatial Discretization of the Advective Terms

The energy equation contains both advective and diffusive terms, whose different physical characteristics must be considered when selecting the spatial discretization scheme. Diffusion spreads information in all directions and is therefore discretized using a second-order central scheme. Although central differencing can also be used for advection, it may produce nonphysical oscillations or instability in convection-dominated regions, particularly when the cell Péclet number is large. In such regions, an upwind scheme is generally more appropriate because it respects the direction of information transport.

In the present work, second-order central differencing is used for the advective terms. This choice is appropriate provided that diffusion is sufficiently strong relative to convection.

The advective terms are written in conservative form as spatial derivatives of the products uT and vT. In the present discretization, each product is treated as a single quantity. For a generic cell-centered variable ϕ, the first-order spatial derivatives are approximated using second-order central differences:

$$\left.\frac{\partial \phi}{\partial x}\right|_{ij} \approx \frac{\phi_{i+1,j} - \phi_{i-1,j}}{2\Delta x}, \tag{11}$$

$$\left.\frac{\partial \phi}{\partial y}\right|_{ij} \approx \frac{\phi_{i,j+1} - \phi_{i,j-1}}{2\Delta y}. \tag{12}$$

Applying these expressions directly to the products uT and vT gives

$$\left.\frac{\partial (uT)}{\partial x}\right|_{ij} \approx \frac{u_{i+1,j}T_{i+1,j} - u_{i-1,j}T_{i-1,j}}{2\Delta x}, \tag{13}$$

and

$$\left.\frac{\partial (vT)}{\partial y}\right|_{ij} \approx \frac{v_{i,j+1}T_{i,j+1} - v_{i,j-1}T_{i,j-1}}{2\Delta y}. \tag{14}$$

For completeness, the cell-centered temperature can also be interpolated to the control-volume faces using second-order central interpolation:

$$T_{i+\frac{1}{2},j} \approx \frac{T_{i,j} + T_{i+1,j}}{2}, \qquad T_{i-\frac{1}{2},j} \approx \frac{T_{i-1,j} + T_{i,j}}{2}, \tag{15}$$

$$T_{i,j+\frac{1}{2}} \approx \frac{T_{i,j} + T_{i,j+1}}{2}, \qquad T_{i,j-\frac{1}{2}} \approx \frac{T_{i,j-1} + T_{i,j}}{2}. \tag{16}$$

The same central interpolation can be applied to other cell-centered variables when face values are required. However, Equations (13) and (14) define the specific advective discretization adopted here: central differences are applied directly to the products uT and vT.

2.4 Spatial Discretization of the Diffusive Term

The diffusive contribution is discretized using second-order central differences. The first derivatives of temperature are approximated as

$$\left.\frac{\partial T}{\partial x}\right|_{ij} \approx \frac{T_{i+1,j} - T_{i-1,j}}{2\Delta x}, \tag{17}$$

$$\left.\frac{\partial T}{\partial y}\right|_{ij} \approx \frac{T_{i,j+1} - T_{i,j-1}}{2\Delta y}. \tag{18}$$

The second derivatives are approximated by

$$\left.\frac{\partial^2 T}{\partial x^2}\right|_{ij} \approx \frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{\Delta x^2}, \tag{19}$$

$$\left.\frac{\partial^2 T}{\partial y^2}\right|_{ij} \approx \frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{\Delta y^2}. \tag{20}$$

Consequently, the Laplacian of temperature is discretized as

$$\begin{aligned}\nabla^2 T|_{ij} \approx \frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{\Delta x^2} \\ + \frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{\Delta y^2}.\end{aligned} \tag{21}$$

2.5 Spatial Discretization of the Viscous-Dissipation Source

The viscous-dissipation source term is evaluated using velocity gradients at the cell centers. The derivatives are approximated using second-order central differences:

$$\left.\frac{\partial u}{\partial x}\right|_{ij} \approx \frac{u_{i+1,j} - u_{i-1,j}}{2\Delta x}, \tag{22}$$

$$\left.\frac{\partial v}{\partial y}\right|_{ij} \approx \frac{v_{i,j+1} - v_{i,j-1}}{2\Delta y}, \tag{23}$$

$$\left.\frac{\partial u}{\partial y}\right|_{ij} \approx \frac{u_{i,j+1} - u_{i,j-1}}{2\Delta y}, \tag{24}$$

$$\left.\frac{\partial v}{\partial x}\right|_{ij} \approx \frac{v_{i+1,j} - v_{i-1,j}}{2\Delta x}. \tag{25}$$

Substituting these expressions into the source-term equation gives

$$\begin{aligned}S_{ij} = \frac{\mathrm{Ec}}{\mathrm{Re}}\Bigg[2\left(\frac{u_{i+1,j} - u_{i-1,j}}{2\Delta x}\right)^2 \\ + 2\left(\frac{v_{i,j+1} - v_{i,j-1}}{2\Delta y}\right)^2 \\ + \left(\frac{u_{i,j+1} - u_{i,j-1}}{2\Delta y} + \frac{v_{i+1,j} - v_{i-1,j}}{2\Delta x}\right)^2\Bigg].\end{aligned} \tag{26}$$

2.6 Complete Spatially Discretized Equation

Combining the central-difference approximations for the advective and diffusive terms with the discretized viscous-dissipation source gives the complete spatially discretized energy equation:

$$\begin{aligned}\frac{d\overline{T}_{ij}}{dt} = & -\frac{1}{2\Delta x}\left(u_{i+1,j}T_{i+1,j} - u_{i-1,j}T_{i-1,j}\right) \\ & + \frac{1}{\mathrm{Re}\,\mathrm{Pr}}\frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{\Delta x^2} \\ & - \frac{1}{2\Delta y}\left(v_{i,j+1}T_{i,j+1} - v_{i,j-1}T_{i,j-1}\right) \\ & + \frac{1}{\mathrm{Re}\,\mathrm{Pr}}\frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{\Delta y^2} + S_{ij} \\ = & \; FI_{ij} + S_{ij} = RHS_{ij}.\end{aligned} \tag{27}$$

Here, S_{ij} denotes the discretized viscous-dissipation source term. Equation (27) is the spatially discretized, semi-discrete energy equation. All spatial derivatives have been replaced by algebraic expressions, whereas the temporal derivative remains continuous. The resulting system of ordinary differential equations can then be advanced in time using an appropriate time-integration method. The temporal discretization and the numerical solution methods used in this project are discussed in the next section.

For the parameter study, the spatially discretized energy equation is reformulated in terms of the Peclet number Pe, the Brinkman number Br, and the convection coefficient c. Using the definitions

$$Pe = \mathrm{Re}\,\mathrm{Pr}, \qquad Br = Ec\,\mathrm{Pr},$$

the coefficient of the viscous-dissipation term becomes

$$\frac{Ec}{\mathrm{Re}} = \frac{Br}{Pe}.$$

The parameterized equation therefore takes the form:

$$\begin{aligned}\frac{d\overline{T}_{ij}}{dt} = & -\frac{c}{2\Delta x}\left(u_{i+1,j}T_{i+1,j} - u_{i-1,j}T_{i-1,j}\right) \\ & - \frac{c}{2\Delta y}\left(v_{i,j+1}T_{i,j+1} - v_{i,j-1}T_{i,j-1}\right) \\ & + \frac{1}{Pe}\left[\frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{\Delta x^2}\right. \\ & \left. + \frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{\Delta y^2}\right] + \frac{Br}{Pe}\,\Phi_{ij} \\ = & \; FI_{ij}(Pe, c) + \frac{Br}{Pe}\,\Phi_{ij} = RHS_{ij},\end{aligned} \tag{28}$$

where Φ_{ij} denotes the discretized viscous-dissipation function without its multiplicative coefficient. The operator $FI_{ij}(Pe, c)$ contains the advective and diffusive contributions. This formulation provides the basis for the parameter study, in which the effects of convection, thermal diffusion, and viscous dissipation are examined through controlled variations of c, Pe, and Br.

2.7 Implementation of Boundary Conditions Using Ghost Cells

The spatial discretization requires temperature values at neighboring cells, including near the boundaries of the computational domain. However, the physical domain does not contain cells outside its boundaries. To apply the same discrete spatial operators to boundary cells, an additional layer of fictitious cells, known as *ghost cells*, is introduced around the physical domain.

The temperature values in ghost cells are not independent unknowns. Instead, they are calculated from the prescribed boundary conditions and the temperature values in adjacent physical cells. This approach allows the discretized equations to retain a consistent stencil near the boundaries.

In the following expressions, the physical cells are indexed by $i = 1, \ldots, n_x$ and $j = 1, \ldots, n_y$. The inlet, bottom-wall, top-wall, and outlet ghost cells are located at $i = 0$, $j = 0$, $j = n_y + 1$, and $i = n_x + 1$, respectively.

2.7.1 Dirichlet Boundary Conditions

For a prescribed temperature at a boundary face, the ghost-cell temperature is determined by assuming that the boundary-face temperature is the average of the adjacent physical-cell and ghost-cell temperatures. At the inlet, this relation is

$$T_{\text{in},j} = \frac{T_{0,j} + T_{1,j}}{2}, \tag{29}$$

which gives

$$\boxed{T_{0,j} = 2T_{\text{in},j} - T_{1,j}.} \tag{30}$$

Similarly, the bottom and top walls have prescribed temperatures T_{bottom} and T_{top}. The corresponding ghost-cell relations are

$$\boxed{T_{i,0} = 2T_{\text{bottom}} - T_{i,1},} \tag{31}$$

and

$$\boxed{T_{i,n_y+1} = 2T_{\text{top}} - T_{i,n_y}.} \tag{32}$$

For the present problem, the inlet temperature varies linearly with the wall-normal coordinate, while the bottom and top wall temperatures are zero and one, respectively:

$$T_{\text{in}}(y) = y, \qquad T_{\text{bottom}} = 0, \qquad T_{\text{top}} = 1. \tag{33}$$

Consequently, the ghost-cell values simplify to

$$T_{0,j} = 2y_j - T_{1,j}, \tag{34}$$

$$T_{i,0} = -T_{i,1}, \tag{35}$$

$$T_{i,n_y+1} = 2 - T_{i,n_y}. \tag{36}$$

2.7.2 Outlet Boundary Condition

At the outlet, a zero streamwise temperature gradient is imposed:

$$\left.\frac{\partial T}{\partial x}\right|_{x=L_x} = 0. \tag{37}$$

Using a first-order approximation to the gradient between the last physical cell and its ghost cell gives

$$\frac{T_{n_x+1,j} - T_{n_x,j}}{\Delta x} = 0. \tag{38}$$

Therefore, the outlet ghost-cell temperature is

$$\boxed{T_{n_x+1,j} = T_{n_x,j}.} \tag{39}$$

The ghost-cell relations are applied whenever the spatial operator is evaluated. They provide the temperature values required by the discrete equations near the boundaries without introducing additional independent unknowns. In the subsequent numerical solution methods, the same boundary conditions are retained, while their algebraic contributions are incorporated according to the structure of each solver.

2.7.3 Boundary Conditions in Delta Form

In the approximate factorization method, the unknown is the temperature increment between consecutive time levels, defined as

$$\delta T_{i,j}^{n+1} = T_{i,j}^{n+1} - T_{i,j}^{n}. \tag{40}$$

The ghost-cell relations must therefore also be expressed in terms of these increments. Since the prescribed inlet and wall temperatures are constant in time, their increments at the boundary faces are zero. Applying the ghost-cell relations at successive time levels and subtracting them gives the corresponding incremental boundary conditions.

At the inlet,

$$\begin{aligned} T_{0,j}^{n+1} &= 2T_{\text{in},j} - T_{1,j}^{n+1}, \\ T_{0,j}^{n} &= 2T_{\text{in},j} - T_{1,j}^{n}, \end{aligned} \tag{41}$$

and consequently,

$$\boxed{\delta T_{0,j}^{n+1} = -\delta T_{1,j}^{n+1}.} \tag{42}$$

Similarly, for the bottom and top walls,

$$\boxed{\delta T_{i,0}^{n+1} = -\delta T_{i,1}^{n+1},} \tag{43}$$

and

$$\boxed{\delta T_{i,n_y+1}^{n+1} = -\delta T_{i,n_y}^{n+1}.} \tag{44}$$

At the outlet, the zero-gradient condition gives

$$T^{n+1}_{n_x+1,j} = T^{n+1}_{n_x,j}, \qquad T^{n}_{n_x+1,j} = T^{n}_{n_x,j}, \tag{45}$$

so that

$$\boxed{\delta T^{n+1}_{n_x+1,j} = \delta T^{n+1}_{n_x,j}.} \tag{46}$$

These incremental relations allow the spatial operator to be evaluated consistently in terms of the unknown temperature updates. They also ensure that the prescribed inlet and wall temperatures remain unchanged after each update, while the zero-gradient condition continues to hold at the outlet.

3. Temporal Discretization

3.1 Approximate Factorization

3.1.1 Implicit Euler Discretization

Following the spatial discretization presented in the previous section, the energy equation can be expressed as a system of ordinary differential equations:

$$\frac{dT_{ij}}{dt} = FI_{ij} + S_{ij} = RHS_{ij}, \tag{47}$$

where FI_{ij} denotes the combined advective and diffusive contributions, and S_{ij} is the viscous-dissipation source term. The cell-centered temperature is denoted by T_{ij}, with the control-volume average represented by the same quantity under the present second-order approximation.

The implicit Euler method approximates the temporal derivative using a backward difference and evaluates the spatial operator at the next time level:

$$\frac{T_{ij}^{n+1} - T_{ij}^{n}}{\Delta t} = FI_{ij}^{n+1} + S_{ij}^{n+1}. \tag{48}$$

Here, n and $n+1$ denote the current and next time levels, respectively, and Δt is the time step. Since the velocity field is prescribed in the present problem, the viscous-dissipation source term is independent of temperature and can be evaluated directly from the velocity gradients.

Substituting the spatially discretized expression into Equation (48) gives

$$\begin{aligned} \frac{T_{ij}^{n+1} - T_{ij}^{n}}{\Delta t} = & - \frac{u_{i+1,j}T_{i+1,j}^{n+1} - u_{i-1,j}T_{i-1,j}^{n+1}}{2\Delta x} \\ & + \frac{1}{\mathrm{Re}\,\mathrm{Pr}} \frac{T_{i+1,j}^{n+1} - 2T_{ij}^{n+1} + T_{i-1,j}^{n+1}}{\Delta x^2} \\ & - \frac{v_{i,j+1}T_{i,j+1}^{n+1} - v_{i,j-1}T_{i,j-1}^{n+1}}{2\Delta y} \\ & + \frac{1}{\mathrm{Re}\,\mathrm{Pr}} \frac{T_{i,j+1}^{n+1} - 2T_{ij}^{n+1} + T_{i,j-1}^{n+1}}{\Delta y^2} + S_{ij}. \end{aligned} \tag{49}$$

Although the implicit Euler method provides a direct temporal discretization, Equation (49) couples the unknown temperatures at the next time level. Solving the resulting system simultaneously for all control volumes can be computationally expensive. The approximate factorization method addresses this difficulty by splitting the two-dimensional operator into directional operators, allowing the solution to be obtained through successive one-dimensional solves.

3.1.2 Delta Formulation

To formulate the approximate factorization method, the temperature update between successive time levels is defined as

$$\delta T_{ij}^{n+1} = T_{ij}^{n+1} - T_{ij}^{n}. \tag{50}$$

Thus, the temperature at the next time level can be written as

$$T_{ij}^{n+1} = T_{ij}^{n} + \delta T_{ij}^{n+1}. \tag{51}$$

Substituting Equation (51) into the implicit Euler equation and separating the terms involving the unknown update from those evaluated at the current time level yields

$$\frac{\delta T_{ij}^{n+1}}{\Delta t} + \left(\mathcal{X}\delta T^{n+1}\right)_{ij} + \left(\mathcal{Y}\delta T^{n+1}\right)_{ij} = RHS_{ij}^{n}, \tag{52}$$

where

$$RHS_{ij}^{n} = FI_{ij}^{n} + S_{ij}. \tag{53}$$

Here, FI_{ij}^{n} is evaluated using the temperature field at time level n, and S_{ij} is the prescribed viscous-dissipation source term.

The directional operators $\mathcal{X}$ and $\mathcal{Y}$ contain the advective and diffusive contributions in the x- and y-directions, respectively. They are defined by

$$\begin{aligned}(\mathcal{X}T)_{ij} = &\frac{u_{i+1,j}T_{i+1,j} - u_{i-1,j}T_{i-1,j}}{2\Delta x} \\ &- \frac{1}{\mathrm{Re}\,\mathrm{Pr}}\frac{T_{i+1,j} - 2T_{ij} + T_{i-1,j}}{\Delta x^2},\end{aligned} \tag{54}$$

and

$$\begin{aligned}(\mathcal{Y}T)_{ij} = &\frac{v_{i,j+1}T_{i,j+1} - v_{i,j-1}T_{i,j-1}}{2\Delta y} \\ &- \frac{1}{\mathrm{Re}\,\mathrm{Pr}}\frac{T_{i,j+1} - 2T_{ij} + T_{i,j-1}}{\Delta y^2}.\end{aligned} \tag{55}$$

With these definitions, the non-source part of the spatial operator satisfies

$$FI_{ij} = -\left(\mathcal{X}T\right)_{ij} - \left(\mathcal{Y}T\right)_{ij}. \tag{56}$$

Multiplying Equation (52) by Δt gives

$$\left(I + \Delta t\,\mathcal{X} + \Delta t\,\mathcal{Y}\right)\delta T^{n+1} = \Delta t\, RHS^{n}, \tag{57}$$

where I denotes the identity operator. This equation is the operator form of the implicit Euler update. Directly solving it requires a coupled two-dimensional system because both directional operators act on the unknown temperature update.

3.1.3 Approximate Factorization

The approximate factorization method replaces the combined operator in Equation (57) with a product of two directional operators:

$$\boxed{\left(I + \Delta t\,\mathcal{X}\right)\left(I + \Delta t\,\mathcal{Y}\right)\delta T^{n+1} = \Delta t\, RHS^{n}.} \tag{58}$$

Expanding the product on the left-hand side gives

$$\Big[I + \Delta t\,\mathcal{X} + \Delta t\,\mathcal{Y} + \Delta t^2\mathcal{X}\mathcal{Y}\Big]\delta T^{n+1} = \Delta t\,RHS^n. \tag{59}$$

Comparing Equation (59) with Equation (57) shows that the factorized formulation introduces the additional cross term

$$\Delta t^2\mathcal{X}\mathcal{Y}\delta T^{n+1}. \tag{60}$$

This term is proportional to Δt^2. The approximation is therefore reasonable when the time step is sufficiently small for the additional factorization term to remain acceptable. However, the temporal accuracy of the underlying implicit Euler method remains first-order.

The main advantage of this approximation is computational. Instead of solving one coupled two-dimensional system, the factorized equation can be solved through two successive directional systems. This procedure is commonly implemented using an alternating-direction implicit (ADI) strategy.

3.1.4 Alternating-Direction Implicit Solution

To solve Equation (58) sequentially, an intermediate temperature update $\widetilde{\delta T}^{n+1}$ is introduced as

$$\widetilde{\delta T}^{n+1} = (I + \Delta t\,\mathcal{Y})\,\delta T^{n+1}. \tag{61}$$

Substituting this definition into Equation (58) yields the first directional system:

$$\boxed{(I + \Delta t\,\mathcal{X})\,\widetilde{\delta T}^{n+1} = \Delta t\,RHS^n.} \tag{62}$$

After solving for the intermediate update, the second directional system is solved:

$$\boxed{(I + \Delta t\,\mathcal{Y})\,\delta T^{n+1} = \widetilde{\delta T}^{n+1}.} \tag{63}$$

The first step treats the x-direction operator implicitly, while the second treats the y-direction operator implicitly. Since each directional operator couples a cell only to its immediate neighbors along one coordinate direction, the resulting linear systems have tridiagonal coefficient matrices along the respective grid lines. These systems can be solved efficiently using a tridiagonal matrix algorithm.

Finally, after obtaining the temperature update, the temperature field is advanced according to

$$\boxed{T_{ij}^{n+1} = T_{ij}^{n} + \delta T_{ij}^{n+1}.} \tag{64}$$

The approximate factorization method thus reduces the solution of a two-dimensional implicit problem to two successive one-dimensional solves. When the objective is to obtain a steady-state temperature field, this procedure can be used for pseudo-time marching until the residual falls below the prescribed convergence tolerance.

Treatment of Boundary Conditions The ghost-cell relations derived in Section 2.7, including their incremental forms, are incorporated into the directional systems before they are solved. Since the unknown is the temperature increment δT^{n+1}, the prescribed inlet and wall temperatures lead to homogeneous incremental boundary relations, whereas the zero-gradient outlet condition requires the ghost-cell increment to equal the adjacent physical-cell increment. Substituting these relations into the directional operators eliminates ghost-cell increments as independent unknowns and modifies the coefficients of the corresponding tridiagonal systems. After solving the two directional systems successively, the temperature field is updated using

$$T_{ij}^{n+1} = T_{ij}^{n} + \delta T_{ij}^{n+1}. \tag{65}$$

3.2 Explicit Runge–Kutta Methods

The spatial discretization of the energy equation yields a system of ordinary differential equations of the form

$$\frac{dT_{ij}}{dt} = RHS_{ij}(T) = FI_{ij}(T) + S_{ij}, \tag{66}$$

where $FI_{ij}(T)$ represents the discretized advective and diffusive contributions, and S_{ij} is the viscous-dissipation source term. For convenience, the complete right-hand side is denoted by the operator $\mathcal{F}$:

$$\mathcal{F}_{ij}(T) = FI_{ij}(T) + S_{ij}. \tag{67}$$

The Runge–Kutta methods advance the solution explicitly by evaluating this operator at several intermediate stages within each time step. Unlike the approximate factorization method, they do not require the solution of implicit linear systems. The second- and fourth-order methods considered in this project are described below.

3.2.1 Second-Order Runge–Kutta Method

The second-order Runge–Kutta method used in this project is the explicit midpoint method. It evaluates the right-hand side at the beginning of the time step and at an intermediate state located halfway through the step.

First, the initial slope is evaluated using the temperature field at time level n:

$$k_{1,ij} = \mathcal{F}_{ij}(T^{n}). \tag{68}$$

An intermediate temperature field is then constructed using a half time step:

$$T_{ij}^{(1)} = T_{ij}^{n} + \frac{\Delta t}{2} k_{1,ij}. \tag{69}$$

The right-hand side is evaluated again at this intermediate state:

$$k_{2,ij} = \mathcal{F}_{ij}\left(T^{(1)}\right). \tag{70}$$

The solution at the next time level is obtained using the intermediate slope:

$$\boxed{T_{ij}^{n+1} = T_{ij}^{n} + \Delta t\, k_{2,ij}.} \tag{71}$$

The explicit midpoint method is second-order accurate in time. Its use of an intermediate evaluation provides a more accurate approximation to the evolution of the solution over a time step than the first-order explicit Euler method.

3.2.2 Fourth-Order Runge–Kutta Method

The classical fourth-order Runge–Kutta method evaluates the right-hand side four times during each time step. These evaluations provide estimates of the solution's slope at the beginning, at two intermediate states, and at the end of the time step.

The first stage evaluates the right-hand side at the current temperature field:

$$k_{1,ij} = \mathcal{F}_{ij}(T^n). \tag{72}$$

The second stage evaluates it at an intermediate state predicted using the first slope:

$$k_{2,ij} = \mathcal{F}_{ij}\left(T^n + \frac{\Delta t}{2} k_1\right). \tag{73}$$

The third stage uses the second slope to construct another intermediate state:

$$k_{3,ij} = \mathcal{F}_{ij}\left(T^n + \frac{\Delta t}{2} k_2\right). \tag{74}$$

Finally, the fourth stage evaluates the right-hand side at a state predicted over the full time step:

$$k_{4,ij} = \mathcal{F}_{ij}\left(T^n + \Delta t\, k_3\right). \tag{75}$$

The four stage evaluations are combined to obtain the updated temperature field:

$$\boxed{T_{ij}^{n+1} = T_{ij}^{n} + \frac{\Delta t}{6}\left(k_{1,ij} + 2k_{2,ij} + 2k_{3,ij} + k_{4,ij}\right).} \tag{76}$$

The classical Runge–Kutta method is fourth-order accurate in time for sufficiently smooth solutions. Although it requires four evaluations of the spatial operator per time step, its higher temporal accuracy can be advantageous when the time step is sufficiently small and stability requirements are satisfied.

3.3 Direct Sparse Solution

3.3.1 Linearization of the Spatial Operator

The spatially discretized energy equation can be expressed as a system of ordinary differential equations:

$$\frac{d\mathbf{T}}{dt} = \mathcal{L}(\mathbf{T}), \tag{77}$$

where $\mathbf{T}$ is the vector of cell-centered temperatures and $\mathcal{L}$ denotes the spatial operator, including advection, diffusion, and viscous dissipation.

For the direct solver, the spatial operator is written in the form

$$\begin{aligned}\mathcal{L}_{ij}(T) = & -c\frac{u_{i+1,j}T_{i+1,j} - u_{i-1,j}T_{i-1,j}}{2\Delta x} \\ & -c\frac{v_{i,j+1}T_{i,j+1} - v_{i,j-1}T_{i,j-1}}{2\Delta y} \\ & +\frac{1}{\mathrm{Pe}}\frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{\Delta x^2} \\ & +\frac{1}{\mathrm{Pe}}\frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{\Delta y^2} + S_{ij},\end{aligned} \tag{78}$$

where Pe is the Peclet number, c is the convection coefficient, and S_{ij} is the viscous-dissipation source term. The velocity field and source term are prescribed and independent of temperature. Consequently, the discretized spatial operator is linear in the unknown temperatures, provided that the boundary conditions are linear.

3.3.2 Sparse Matrix Formulation

Collecting the temperatures of the $n_x n_y$ physical cells into a vector gives

$$\boxed{\frac{d\mathbf{T}}{dt} = \mathbf{A}\mathbf{T} + \mathbf{b},} \tag{79}$$

where $\mathbf{A}$ is the sparse matrix representing the linear spatial operator, and $\mathbf{b}$ contains the temperature-independent contributions from the boundary conditions and viscous dissipation.

The matrix is sparse because each discretized equation couples a cell primarily to its immediate neighbors. The physical-cell temperature vector is constructed by flattening the two-dimensional temperature array in C order, with the wall-normal index varying fastest. The corresponding zero-based array indices are mapped to the vector index

$$k(i,j) = in_y + j, \qquad i = 0,\ldots,n_x - 1, \qquad j = 0,\ldots,n_y - 1. \tag{80}$$

The forcing vector can be obtained by evaluating the spatial operator with all physical-cell temperatures set to zero, while retaining the prescribed boundary conditions:

$$\boxed{\mathbf{b} = \mathcal{L}(\mathbf{0}).} \tag{81}$$

The ghost-cell relations derived in Section 2.7 are substituted into the spatially discretized

equations before assembling the coefficient matrix. This eliminates ghost-cell temperatures as independent unknowns. Contributions proportional to physical-cell temperatures are incorporated into the matrix $\mathbf{A}$, while temperature-independent contributions from prescribed boundary values are included in the forcing vector $\mathbf{b}$. Consequently, the resulting linear system contains only the temperatures of the physical cells.

3.3.3 Implicit Euler Time Integration

The semi-discrete system is advanced in pseudo-time using the implicit Euler method:

$$\frac{\mathbf{T}^{n+1} - \mathbf{T}^n}{\Delta t} = \mathbf{A}\mathbf{T}^{n+1} + \mathbf{b}, \tag{82}$$

where Δt denotes the pseudo-time step. Rearranging the equation gives the linear system

$$\boxed{(\mathbf{I} - \Delta t\, \mathbf{A})\, \mathbf{T}^{n+1} = \mathbf{T}^n + \Delta t\, \mathbf{b}.} \tag{83}$$

Defining the coefficient matrix as

$$\mathbf{M} = \mathbf{I} - \Delta t\, \mathbf{A}, \tag{84}$$

the update becomes

$$\mathbf{M}\mathbf{T}^{n+1} = \mathbf{T}^n + \Delta t\, \mathbf{b}. \tag{85}$$

When the grid, pseudo-time step, velocity field, model parameters, and boundary conditions remain fixed, the matrix $\mathbf{M}$ does not change between iterations. Its sparse LU factorization can therefore be computed once and reused for subsequent time steps:

$$\mathbf{M} = \mathbf{L}_{\mathrm{LU}}\mathbf{U}_{\mathrm{LU}}. \tag{86}$$

At each iteration, the right-hand side is first solved using the lower-triangular factor:

$$\mathbf{L}_{\mathrm{LU}}\mathbf{y} = \mathbf{T}^n + \Delta t\, \mathbf{b}, \tag{87}$$

followed by the upper-triangular solve

$$\mathbf{U}_{\mathrm{LU}}\mathbf{T}^{n+1} = \mathbf{y}. \tag{88}$$

This procedure uses a direct linear-system solver at each time step rather than an iterative method to solve the coupled linear equations. Reusing the LU factorization avoids repeating the relatively expensive matrix-factorization stage during the pseudo-time-marching process.

In summary, the direct solver combines a sparse matrix representation of the spatially discretized energy equation with implicit Euler pseudo-time marching and a reusable sparse LU factorization. This approach provides a reference numerical solution against which the approximate factorization and explicit Runge–Kutta methods can be compared.

3.4 Convergence Assessment

The objective of the numerical time-marching methods is to obtain a steady-state temperature distribution. At steady state, the temporal derivative vanishes, and the spatially discretized right-hand side approaches zero:

$$RHS_{ij}(T) = FI_{ij}(T) + S_{ij} = 0. \tag{89}$$

Convergence is assessed using the root-mean-square (RMS) residual of the spatially discretized energy equation over the physical cells:

$$\boxed{R_{\mathrm{RMS}} = \sqrt{\frac{1}{n_x n_y} \sum_{i=1}^{n_x} \sum_{j=1}^{n_y} [RHS_{ij}(T)]^2.}} \tag{90}$$

This residual provides a common convergence measure for all numerical solution methods considered in this project, since it evaluates how closely the current temperature field satisfies the same discrete steady-state energy equation, independently of the temporal integration scheme.

The calculation is terminated when the RMS residual falls below a prescribed tolerance or the maximum number of iterations is reached. The residual is therefore used to assess convergence toward a steady-state numerical solution. However, a small residual does not by itself quantify the spatial discretization error or establish grid independence; these require separate verification studies.

4. Temporal Accuracy and Stability Analysis

4.1 Semi-Discrete System and Eigenvalue Analysis

After spatial discretization and implementation of the boundary conditions, the energy equation can be expressed as a system of ordinary differential equations:

$$\boxed{\frac{d\mathbf{T}}{dt} = \mathbf{A}\mathbf{T} + \mathbf{b},} \tag{91}$$

where $\mathbf{T}$ is the vector of physical-cell temperatures, $\mathbf{A}$ is the assembled spatial-operator matrix, and $\mathbf{b}$ contains the temperature-independent contributions from prescribed boundary values and viscous dissipation.

The matrix $\mathbf{A}$ depends on the spatial discretization, grid resolution, prescribed velocity field, Peclet number, convection coefficient, and implementation of the boundary conditions. Since the computational domain has non-periodic boundary conditions and a streamwise velocity that varies across the channel, the eigenvalues cannot generally be obtained using the Fourier-mode formula for a periodic, constant-coefficient stencil.

Instead, the spatial matrix must first be assembled using the actual discretized equations and boundary conditions. Its eigenvalues are then calculated numerically from

$$\boxed{\mathbf{A}\mathbf{q}_k = \lambda_k \mathbf{q}_k, \qquad k = 1, \ldots, N,} \tag{92}$$

where λ_k and $\mathbf{q}_k$ are the eigenvalues and corresponding eigenvectors, and $N = n_x n_y$ is the number of physical-cell temperature unknowns.

The eigenvalues characterize the modal behaviour of the linear spatial operator and provide the basis for examining the stability of the time-integration methods. Since the source vector $\mathbf{b}$ is independent of temperature, it does not directly enter the amplification factors derived below.

4.2 Amplification Factors and Temporal Accuracy

Consider the scalar linear equation

$$\frac{dw}{dt} = \lambda w, \tag{93}$$

where λ represents an eigenvalue of the spatial operator. Define the dimensionless parameter

$$z = \lambda \Delta t, \tag{94}$$

where Δt is the time step.

The exact solution advances according to

$$w^{n+1} = e^z w^n. \tag{95}$$

A numerical time-integration method produces an update of the form

$$w^{n+1} = G(z) w^n, \tag{96}$$

where $G(z)$ is the amplification factor of the method.

Comparing the amplification factor with the Taylor expansion of the exact solution,

$$e^z = 1 + z + \frac{z^2}{2!} + \frac{z^3}{3!} + \frac{z^4}{4!} + \cdots, \tag{97}$$

provides a way to determine the temporal order of accuracy. A method is of order p when its amplification factor matches the expansion of e^z through the term z^p, with the first unmatched term having order z^{p+1}.

4.2.1 Implicit Euler

The implicit Euler method evaluates the spatial operator at the next time level:

$$\frac{w^{n+1} - w^n}{\Delta t} = \lambda w^{n+1}. \tag{98}$$

Rearranging gives

$$w^{n+1} = \frac{1}{1 - \lambda \Delta t} w^n. \tag{99}$$

Therefore, its amplification factor is

$$\boxed{G_{\mathrm{IE}}(z) = \frac{1}{1 - z}.} \tag{100}$$

Expanding this expression as a power series gives

$$G_{\mathrm{IE}}(z) = 1 + z + z^2 + z^3 + \cdots. \tag{101}$$

Comparison with Equation (97) shows agreement only through the linear term. Implicit Euler is therefore first-order accurate in time, with a global temporal error of order $\mathcal{O}(\Delta t)$ for sufficiently smooth solutions.

The absolute-stability condition is

$$\left| \frac{1}{1 - \lambda \Delta t} \right| \leq 1. \tag{102}$$

If $\mathrm{Re}(\lambda) \leq 0$, then

$$\begin{aligned} |1 - \lambda \Delta t|^2 &= 1 - 2\Delta t \ \mathrm{Re}(\lambda) + \Delta t^2 |\lambda|^2 \\ &\geq 1. \end{aligned} \tag{103}$$

Consequently, implicit Euler satisfies the absolute-stability condition for every positive time step when the eigenvalues of the spatial operator lie in the non-positive real half-plane. This property is known as A-stability.

However, A-stability does not guarantee that an arbitrarily large time step will produce an accurate transient solution or efficiently converge to a steady state. The spectrum of the actual spatial matrix and the practical convergence behaviour must still be considered.

4.2.2 Second-Order Runge–Kutta Method

The explicit midpoint method advances the solution through an intermediate state:

$$w^{(1)} = w^n + \frac{\Delta t}{2}\lambda w^n, \tag{104}$$

$$w^{n+1} = w^n + \Delta t\, \lambda w^{(1)}. \tag{105}$$

Substituting the intermediate value into the final update gives

$$w^{n+1} = \left(1 + \lambda\Delta t + \frac{(\lambda\Delta t)^2}{2}\right) w^n. \tag{106}$$

The amplification factor is therefore

$$\boxed{G_{\mathrm{RK2}}(z) = 1 + z + \frac{z^2}{2}.} \tag{107}$$

This polynomial matches the Taylor expansion of the exact amplification factor through the quadratic term. The explicit midpoint method is consequently second-order accurate in time, with global temporal error $\mathcal{O}(\Delta t^2)$ for sufficiently smooth solutions.

Its absolute-stability condition is

$$\boxed{\left|1 + \lambda_k\Delta t + \frac{(\lambda_k\Delta t)^2}{2}\right| \le 1 \qquad \text{for every } k.} \tag{108}$$

Since this condition is not satisfied for arbitrary complex values of $\lambda_k\Delta t$, the explicit midpoint method is conditionally stable. The allowable time step depends on the eigenvalues of the assembled spatial matrix, which in turn depend on the grid, velocity field, Peclet number, and convection coefficient.

4.2.3 Fourth-Order Runge–Kutta Method

The classical fourth-order Runge–Kutta method evaluates the right-hand side at four stages:

$$k_1 = \lambda w^n, \tag{109}$$

$$k_2 = \lambda\left(w^n + \frac{\Delta t}{2}k_1\right), \tag{110}$$

$$k_3 = \lambda\left(w^n + \frac{\Delta t}{2}k_2\right), \tag{111}$$

$$k_4 = \lambda\left(w^n + \Delta t\, k_3\right). \tag{112}$$

The solution is updated according to

$$w^{n+1} = w^n + \frac{\Delta t}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right). \tag{113}$$

Substitution of the four stages yields the amplification factor

$$\boxed{G_{\mathrm{RK4}}(z) = 1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24}.} \tag{114}$$

This polynomial matches the Taylor expansion of the exact amplification factor through the fourth-order term. The classical Runge–Kutta method is therefore fourth-order accurate in time, with global temporal error $\mathcal{O}(\Delta t^4)$ for sufficiently smooth solutions.

Its absolute-stability condition is

$$\boxed{\left|1 + \lambda_k \Delta t + \frac{(\lambda_k \Delta t)^2}{2} + \frac{(\lambda_k \Delta t)^3}{6} + \frac{(\lambda_k \Delta t)^4}{24}\right| \leq 1 \qquad \text{for every } k.} \tag{115}$$

Like RK2, classical RK4 is conditionally stable. Its stability region differs from that of RK2, so its allowable time step must be assessed using its own amplification factor rather than inferred from the RK2 stability limit.

4.3 Numerical Assessment of Stability

Once the spatial matrix has been assembled, the eigenvalues can be used to assess the modal stability of the explicit Runge–Kutta methods. For a candidate time step, define

$$G_{\max}(\Delta t) = \max_{k=1,\ldots,N} \left|G(\lambda_k \Delta t)\right|. \tag{116}$$

The candidate time step satisfies the modal absolute-stability criterion if

$$\boxed{G_{\max}(\Delta t) \leq 1.} \tag{117}$$

The maximum stable time step can be estimated by evaluating this condition over candidate time steps and refining the search near the stability boundary. This calculation is performed separately for RK2 and RK4 because their amplification factors differ.

The direct sparse solver uses implicit Euler time integration. Its time integrator is A-stable when the spectrum of the assembled spatial operator lies in the non-positive real half-plane. The direct linear-system solution itself does not introduce a separate explicit time-step stability restriction. Nevertheless, because the solver is used for pseudo-time marching toward steady state, its practical behaviour depends on the selected time step, the properties of the discrete operator, and the numerical conditioning of the linear system. These factors are assessed through numerical convergence tests.

The approximate factorization method requires a separate assessment. Although it is derived from the implicit Euler equation, replacing the combined spatial operator with a product of directional operators changes the numerical update. Its stability therefore cannot be inferred solely from the amplification factor of fully implicit Euler. In this project, the behaviour of the approximate factorization method is evaluated numerically by testing candidate time steps and monitoring the resulting residuals and convergence behaviour.

The stability analysis therefore provides a theoretical basis for choosing time steps, while nu-

merical tests and convergence histories are used to verify the behaviour of the implemented solvers.

5. Numerical Accuracy Assessment

Numerical accuracy must be assessed independently of convergence to a steady state. A small residual indicates that the discretized governing equation is satisfied closely, but it does not necessarily imply that the numerical solution accurately represents the physical problem.

The spatial accuracy of the numerical solution can be assessed by comparing it with an exact solution, when one is available. Otherwise, the observed order of convergence can be estimated by comparing numerical solutions obtained on successively refined grids.

5.1 Accuracy Assessment Using an Exact Solution

When an exact solution is available, the numerical error is calculated by comparing the numerical and exact solutions. For a numerical scheme with spatial order p, the error in the asymptotic range can be expressed as

$$E(h) \approx Ch^p, \tag{118}$$

where h is a characteristic grid spacing and C is a grid-independent constant.

For two grids with spacings h_1 and h_2, the ratio of their errors is

$$\frac{E_1}{E_2} \approx \left(\frac{h_1}{h_2}\right)^p. \tag{119}$$

When the grid spacing is halved, $h_2 = h_1/2$, the observed spatial order can be estimated as

$$\boxed{p \approx \frac{\ln(E_1/E_2)}{\ln 2}.} \tag{120}$$

The error can be quantified using the root-mean-square (RMS) norm of the difference between the numerical and exact temperature fields:

$$E = \sqrt{\frac{1}{N_{\mathrm{CV}}} \sum_{i=1}^{n_x} \sum_{j=1}^{n_y} [T_{ij} - T_{\mathrm{exact}}(x_i, y_j)]^2}, \tag{121}$$

where $N_{\mathrm{CV}} = n_x n_y$ is the total number of control volumes, and (x_i, y_j) denotes the center of each control volume.

5.2 Grid Convergence Without an Exact Solution

For the present problem, an exact solution for the complete temperature field subject to the prescribed inlet condition is not available. Therefore, the numerical error cannot be calculated directly using Equation (121). Instead, the observed spatial order of convergence is estimated by comparing solutions obtained on three successively refined grids.

Assuming that the numerical solutions approach the same limiting solution as the grid is refined, the solution on a grid with characteristic spacing h can be expressed as

$$T_h = T_{\mathrm{exact}} + Ch^p, \tag{122}$$

where T_{exact} represents the limiting solution, C is the leading error coefficient, and p is the observed spatial order of convergence.

Let T_{M_1}, T_{M_2}, and T_{M_3} denote the numerical solutions on three grids whose characteristic spacings satisfy

$$h_2 = \frac{h_1}{2}, \qquad h_3 = \frac{h_2}{2}. \tag{123}$$

The differences between successive solutions are defined as

$$D_{12} = \|T_{M_1} - T_{M_2}\|_2 , \qquad D_{23} = \|T_{M_2} - T_{M_3}\|_2 , \tag{124}$$

where $\|\cdot\|_2$ denotes the discrete RMS norm. Since the solutions are defined on different cell-centered grids, their temperature fields must first be interpolated onto common physical locations before the differences are calculated.

Assuming that the solutions are in the asymptotic convergence range, the differences between successive grids satisfy

$$\frac{D_{23}}{D_{12}} \approx \frac{1}{2^p}. \tag{125}$$

Taking logarithms gives the observed spatial order:

$$\boxed{p_{\text{obs}} = \frac{\ln(D_{12}/D_{23})}{\ln 2}.} \tag{126}$$

This approach estimates the spatial convergence order without requiring an exact solution. Its reliability depends on the solutions exhibiting consistent convergence and the leading spatial discretization error dominating other numerical errors. Therefore, all grids must be sufficiently converged to steady state so that the remaining pseudo-time convergence error does not significantly affect the differences between solutions.

Finally, the observed order of convergence does not by itself establish the absolute accuracy of the finest-grid solution. It indicates how rapidly the numerical solution approaches its limiting value as the grid is refined.

6. Computational Implementation

The computational implementation is organized into separate procedures for grid generation, initialization of the solution fields, and enforcement of boundary conditions. This separation allows the same computational setup to be used by all numerical solution methods.

6.1 Grid Generation

The computational grid is uniform and cell-centered. The grid generation procedure calculates the grid spacings and the coordinates of the cell centers from the domain dimensions and the number of physical cells in each direction.

Algorithm 1 Grid Generation

Require: Domain dimensions L_x, L_y and cell counts n_x, n_y
Ensure: Cell-center coordinates x_i, y_j and grid spacings $\Delta x, \Delta y$
1: $\Delta x \leftarrow L_x/n_x$
2: $\Delta y \leftarrow L_y/n_y$
3: **for** $i = 0, \ldots, n_x - 1$ **do**
4: $\quad x_i \leftarrow (i + \frac{1}{2})\Delta x$
5: **end for**
6: **for** $j = 0, \ldots, n_y - 1$ **do**
7: $\quad y_j \leftarrow (j + \frac{1}{2})\Delta y$
8: **end for**
9: **return** $x, y, \Delta x, \Delta y$

6.2 Initialization of the Solution Fields

The initial temperature distribution is prescribed as $T(x, y, 0) = y$. The temperature is therefore initialized using the wall-normal coordinate, while the same profile is assigned at every streamwise location. The velocity components are calculated from the prescribed fully developed velocity profile and remain fixed throughout the simulation.

Algorithm 2 Initialization of Temperature and Velocity Fields

Require: Grid coordinates y_j, cell counts n_x, n_y, and mean velocity $\overline{u}$
Ensure: Initial temperature field T and velocity fields u, v
1: Allocate the temperature array, including ghost cells
2: **for** each physical cell (i, j) **do**
3: $\quad T_{ij} \leftarrow y_j$
4: $\quad u_{ij} \leftarrow 6\overline{u}\, y_j(1 - y_j)$
5: $\quad v_{ij} \leftarrow 0$
6: **end for**
7: Apply the temperature boundary conditions
8: **return** T, u, v

6.3 Boundary-Condition Implementation

The temperature array includes one layer of ghost cells around the physical domain. The function `apply_temperature_boundary_conditions` updates these values using the relations

established in Section 2.7. The ghost-cell temperatures are calculated from the adjacent physical-cell temperatures and the prescribed boundary values.

Algorithm 3 Application of Temperature Boundary Conditions

Require: Temperature array T, grid dimensions n_x, n_y, grid spacing Δy, inlet profile $T_{\text{in}}(y)$, and wall temperatures $T_{\text{bottom}}, T_{\text{top}}$
Ensure: Updated temperature array T
for each streamwise index i, including ghost-cell indices **do**
  $T_{i,0} \leftarrow 2T_{\text{bottom}} - T_{i,1}$
  $T_{i,n_y+1} \leftarrow 2T_{\text{top}} - T_{i,n_y}$
end for
for $j = 1, \ldots, n_y$ **do**
  $y_j \leftarrow (j - \frac{1}{2})\Delta y$
  $T_{0,j} \leftarrow 2T_{\text{in}}(y_j) - T_{1,j}$
  $T_{n_x+1,j} \leftarrow T_{n_x,j}$
end for
return T

The same boundary-condition procedure is reused during the numerical solution whenever updated temperature values require consistent ghost-cell values. The ghost cells are not independent unknowns; they provide the values needed to evaluate the spatial operator near the boundaries.

6.4 Convergence Monitoring and Diagnostics

Two complementary quantities are used to assess the numerical solutions: the normalized error relative to a reference temperature field and the residual of the discrete steady energy equation. The error measures the difference between two temperature fields, whereas the residual measures how closely the computed solution satisfies the discretized governing equation.

6.4.1 Normalized L_p Error

The error between the numerical and reference solutions is defined for each control volume as

$$E_i = T_i - T_i^{\text{ref}}, \tag{127}$$

where T_i and T_i^{ref} denote the numerical and reference temperatures in control volume i, respectively. A normalized, area-weighted L_p error measure is defined as

$$L_p = \left(\frac{\sum_{i=1}^{N_{cv}} A_i |E_i|^p}{\sum_{i=1}^{N_{cv}} A_i} \right)^{1/p}, \tag{128}$$

where N_{cv} is the number of physical control volumes, A_i is the area of control volume i, and p specifies the error measure. The denominator represents the total computational domain area, so the expression measures an area-weighted average of the error magnitude raised to the power p, followed by the corresponding root.

For $p = 2$, Equation (128) becomes

$$L_2 = \sqrt{\frac{\sum_{i=1}^{N_{cv}} A_i |E_i|^2}{\sum_{i=1}^{N_{cv}} A_i}}. \tag{129}$$

This measure is the area-weighted root-mean-square (RMS) error. It assigns greater weight to larger local errors because the differences are squared before averaging.

In the present study, the computational grid is uniform, such that every control volume has area $A_i = \Delta x \Delta y$. The cell areas therefore cancel in Equation (129), giving

$$L_2 = \sqrt{\frac{1}{N_{cv}} \sum_{i=1}^{N_{cv}} |E_i|^2}. \tag{130}$$

Accordingly, the error is evaluated in the implementation as the square root of the mean squared difference between the two temperature arrays. Only physical-cell values are included; ghost cells are excluded from the comparison.

Algorithm 4 Normalized L_2 Error Evaluation

Require: Reference temperature field T^{ref} and numerical temperature field T
Ensure: Normalized error L_2
1: Convert both fields to floating-point arrays
2: **if** shape(T^{ref}) $\neq$ shape(T) **then**
3: Raise a shape-mismatch error
4: **end if**
5: $E \leftarrow T - T^{\text{ref}}$
6: $L_2 \leftarrow \sqrt{\text{mean}(|E|^2)}$
7: **return** L_2

This normalized error measure is useful for comparing temperature fields obtained on different uniform grid resolutions because its value is not directly increased merely by increasing the number of control volumes. Its interpretation nevertheless depends on the reference solution and on comparing the fields over the same physical domain.

6.4.2 Steady-State Residual

The steady-state residual is evaluated using the discrete spatial operator implemented in `build_rhs`. For each physical wall-normal index, the function evaluates the right-hand side of the semi-discrete energy equation and stores the resulting values in an auxiliary array. The residual is then extracted over the physical computational domain, excluding ghost cells.

Writing the semi-discrete equation as

$$\frac{\mathrm{d}T_{ij}}{\mathrm{d}t} = \mathcal{R}_{ij}(T), \tag{131}$$

the steady-state residual at cell (i, j) is $\mathcal{R}_{ij}(T)$, corresponding to the discrete right-hand side

evaluated using the current temperature field. The RMS residual is defined as

$$R_{\mathrm{RMS}} = \sqrt{\frac{1}{n_x n_y} \sum_{i=1}^{n_x} \sum_{j=1}^{n_y} \left[\mathcal{R}_{ij}(T)\right]^2}, \tag{132}$$

where n_x and n_y are the numbers of physical cells in the streamwise and wall-normal directions, respectively.

At steady state, the time derivative vanishes and the discrete right-hand side approaches zero throughout the computational domain. Consequently, a decreasing RMS residual indicates convergence toward a solution of the discrete steady energy equation.

Algorithm 5 Steady-State Residual Evaluation

Require: Mesh dimensions n_x, n_y, temperature field T, velocity fields u, v, Pe, Br, domain dimensions L_x, L_y, and convection coefficient α_c
Ensure: RMS residual R_{RMS}
1: Allocate a zero-filled auxiliary array $\mathcal{R}$
2: **for** $j = 1, \ldots, n_y$ **do**
3: Evaluate the discrete right-hand side using `build_rhs` at wall-normal index j
4: Store the returned values in column j of $\mathcal{R}$
5: **end for**
6: Extract the physical-cell residuals $\mathcal{R}_{\mathrm{phys}} \leftarrow \mathcal{R}_{1:n_x,\,1:n_y}$
7: $R_{\mathrm{RMS}} \leftarrow \sqrt{\mathrm{mean}\left(\mathcal{R}_{\mathrm{phys}}^2\right)}$
8: **return** R_{RMS}

The RMS residual assesses convergence of the discrete governing equation, whereas the normalized L_2 error assesses agreement with a reference temperature field. These quantities are complementary: a small residual indicates that the discrete equation is nearly satisfied, but it does not by itself establish the accuracy of the numerical solution or grid independence.

6.5 Approximate Factorization Method

The approximate factorization (APF) method advances the semi-discrete energy equation in pseudo-time by splitting the two-dimensional update into two successive one-dimensional linear systems. This approach avoids solving a single multidimensional system at every iteration. Instead, the resulting systems are tridiagonal and can be solved efficiently using the Thomas algorithm.

6.5.1 Approximate Factorization Formulation

Let the temperature correction during one pseudo-time step be defined as

$$\delta T_{ij} = T_{ij}^{n+1} - T_{ij}^{n}, \tag{133}$$

where n and $n+1$ denote the current and next pseudo-time levels, respectively.

The semi-discrete energy equation is written as

$$\frac{\mathrm{d}T_{ij}}{\mathrm{d}t} = \mathcal{L}_x(T)_{ij} + \mathcal{L}_y(T)_{ij} + S_{ij} = \mathcal{R}_{ij}(T), \tag{134}$$

where $\mathcal{L}_x$ and $\mathcal{L}_y$ represent the streamwise and wall-normal transport operators, respectively, and S_{ij} is the viscous-dissipation source term. The spatial derivatives are approximated using the central finite-difference expressions introduced in the previous section.

For compactness, the two spatial operators are expressed as

$$\begin{aligned}\mathcal{L}_x(T)_{ij} = &-\frac{\alpha_c\,(u_{i+1,j}T_{i+1,j} - u_{i-1,j}T_{i-1,j})}{2\Delta x} \\ &+ \frac{1}{\mathrm{Pe}}\frac{T_{i+1,j} - 2T_{ij} + T_{i-1,j}}{\Delta x^2},\end{aligned} \tag{135}$$

$$\begin{aligned}\mathcal{L}_y(T)_{ij} = &-\frac{\alpha_c\,(v_{i,j+1}T_{i,j+1} - v_{i,j-1}T_{i,j-1})}{2\Delta y} \\ &+ \frac{1}{\mathrm{Pe}}\frac{T_{i,j+1} - 2T_{ij} + T_{i,j-1}}{\Delta y^2},\end{aligned} \tag{136}$$

where

$$\mathrm{Pe} = \mathrm{Re}\,\mathrm{Pr}, \qquad \mathrm{Br} = \mathrm{Ec}\,\mathrm{Pr}, \tag{137}$$

and α_c is the convection coefficient used in the implementation, with default value $\alpha_c = 1$.

The viscous-dissipation source term is evaluated from the prescribed velocity field:

$$\boxed{S_{ij} = \frac{\mathrm{Br}}{\mathrm{Pe}}\left[2\left(\frac{\partial u}{\partial x}\right)^2 + 2\left(\frac{\partial v}{\partial y}\right)^2 + \left(\frac{\partial v}{\partial x} + \frac{\partial u}{\partial y}\right)^2\right]_{ij}.} \tag{138}$$

The velocity gradients are approximated using central differences. Since $\mathrm{Br}/\mathrm{Pe} = \mathrm{Ec}/\mathrm{Re}$, Equation (138) is consistent with the source term in the governing energy equation.

The residual at the current pseudo-time level is therefore

$$\boxed{\mathcal{R}^n_{ij} = \mathcal{L}_x(T^n)_{ij} + \mathcal{L}_y(T^n)_{ij} + S_{ij}.} \tag{139}$$

The APF formulation approximates the multidimensional correction equation through the factorized system

$$\boxed{(I - \Delta t\,\mathcal{L}_x)\,(I - \Delta t\,\mathcal{L}_y)\,\delta T = \Delta t\,\mathcal{R}^n,} \tag{140}$$

where I denotes the identity operator and Δt is the pseudo-time step. The product of the two operators introduces a cross term proportional to Δt^2. Consequently, the factorized equation approximates the corresponding unfactorized multidimensional correction equation.

Introducing an intermediate correction $\delta\widetilde{T}$, the factorized equation is solved in two stages:

$$(I - \Delta t\,\mathcal{L}_x)\,\delta\widetilde{T} = \Delta t\,\mathcal{R}^n, \tag{141}$$

$$(I - \Delta t\,\mathcal{L}_y)\,\delta T = \delta\widetilde{T}. \tag{142}$$

The first stage solves for the intermediate correction along the streamwise direction. The second

stage uses this intermediate field to obtain the final correction along the wall-normal direction. The temperature is then updated as

$$\boxed{T_{ij}^{n+1} = T_{ij}^{n} + \delta T_{ij}.} \tag{143}$$

6.5.2 Construction of the Spatial Residual

The right-hand side of the first factorized system is constructed using the function `build_rhs`. For each physical wall-normal index j, this function evaluates the streamwise and wall-normal convective and diffusive contributions, calculates the viscous-dissipation source, and assembles the residual vector.

Defining the transport contribution as

$$F_{ij} = \mathcal{L}_x(T)_{ij} + \mathcal{L}_y(T)_{ij}, \tag{144}$$

the residual is

$$\mathcal{R}_{ij}(T) = F_{ij} + S_{ij}. \tag{145}$$

The source term is calculated using Equation (138). The resulting algorithm corresponds to the implementation of `build_rhs`.

Algorithm 6 Construction of the Spatial Residual Vector

Require: Mesh dimensions n_x, n_y, wall-normal index j, temperature field T, velocity fields u, v, parameters Pe and Br, domain dimensions L_x, L_y, and convection coefficient α_c
Ensure: Residual vector $\mathbf{r}^{(j)}$

```
Δx ← L_x / n_x
Δy ← L_y / n_y
Initialize r^(j) as a zero vector of length n_x + 2
for i = 1, ..., n_x do
    Evaluate the transport contribution F_ij using Equation (144)
    Calculate the four velocity gradients using central differences
    Evaluate the source term S_ij using Equation (138)
    r_i^(j) ← F_ij + S_ij
end for
return r^(j)
```

The returned vector contains residual values at physical streamwise indices; its ghost-cell entries remain zero. Accordingly, the right-hand side of the first factorized system is $\Delta t\, \mathbf{r}^{(j)}$.

6.5.3 Boundary Conditions in the Factorized Systems

The boundary conditions are imposed through the boundary rows of the factorized systems. In terms of temperature corrections, the four boundary conditions are

$$\text{Inlet:} \quad \delta T_{0,j} + \delta T_{1,j} = 0, \tag{146}$$

$$\text{Outlet:} \quad \delta T_{n_x,j} - \delta T_{n_x+1,j} = 0, \tag{147}$$

$$\text{Bottom wall:} \quad \delta T_{i,0} + \delta T_{i,1} = 0, \tag{148}$$

$$\text{Top wall:} \quad \delta T_{i,n_y} + \delta T_{i,n_y+1} = 0. \tag{149}$$

The inlet and outlet conditions are incorporated into the streamwise system, while the wall conditions are imposed in the wall-normal system. The corresponding matrix rows contain two nonzero coefficients: $(1, 1)$ for prescribed-temperature boundaries and $(1, -1)$ for the outlet zero-gradient condition, with the latter ordered as in the matrix implementation.

6.5.4 Streamwise System

For each fixed wall-normal index j, the first APF stage requires the solution of Equation (141) along the streamwise direction. For a physical cell (i, j), the resulting equation is

$$\boxed{a^x_{W,i}\,\delta\widetilde{T}_{i-1,j} + a^x_P\,\delta\widetilde{T}_{ij} + a^x_{E,i}\,\delta\widetilde{T}_{i+1,j} = \Delta t\,\mathcal{R}^n_{ij},} \tag{150}$$

where the coefficients are

$$a^x_{W,i} = -\frac{\alpha_c u_{i-1,j}\Delta t}{2\Delta x} - \frac{\Delta t}{\mathrm{Pe}\,\Delta x^2}, \tag{151}$$

$$a^x_P = 1 + \frac{2\Delta t}{\mathrm{Pe}\,\Delta x^2}, \tag{152}$$

$$a^x_{E,i} = \frac{\alpha_c u_{i+1,j}\Delta t}{2\Delta x} - \frac{\Delta t}{\mathrm{Pe}\,\Delta x^2}. \tag{153}$$

For each j, the unknown corrections are arranged in a vector containing the physical cells and the two streamwise ghost cells. The coefficient matrix is tridiagonal. The right-hand side contains the residual values at the physical cells, scaled by Δt, with zero entries at the ghost indices.

The corresponding matrix is assembled by `build_x_matrix`, as summarized below.

Algorithm 7 Construction of the Streamwise Coefficient Matrix

Require: Mesh dimensions n_x, n_y, wall-normal index j, time step Δt, Peclet number Pe, velocity field u, domain length L_x, and convection coefficient α_c
Ensure: Streamwise matrix $\mathbf{A}_x^{(j)}$

$\Delta x \leftarrow L_x/n_x$
Initialize $\mathbf{A}_x^{(j)}$ as a zero matrix of size $(n_x+2)\times(n_x+2)$
Set the inlet row: $A^x_{0,0} \leftarrow 1$, $A^x_{0,1} \leftarrow 1$
Set the outlet row: $A^x_{n_x+1,n_x} \leftarrow 1$, $A^x_{n_x+1,n_x+1} \leftarrow -1$
for $i = 1, \ldots, n_x$ **do**
    $A^x_{i,i} \leftarrow 1 + \dfrac{2\Delta t}{\mathrm{Pe}\,\Delta x^2}$
    $A^x_{i,i+1} \leftarrow \dfrac{\alpha_c \Delta t\, u_{i+1,j}}{2\Delta x} - \dfrac{\Delta t}{\mathrm{Pe}\,\Delta x^2}$
    $A^x_{i,i-1} \leftarrow -\dfrac{\alpha_c \Delta t\, u_{i-1,j}}{2\Delta x} - \dfrac{\Delta t}{\mathrm{Pe}\,\Delta x^2}$
end for
return $\mathbf{A}_x^{(j)}$

For each physical wall-normal index, the resulting linear system is

$$\boxed{\mathbf{A}_x^{(j)} \delta\widetilde{\mathbf{T}}^{(j)} = \Delta t\, \mathbf{r}^{(j)}.} \tag{154}$$

It is solved using the Thomas algorithm, producing one row of the intermediate correction field.

6.5.5 Wall-Normal System

After the intermediate correction has been obtained, the second APF stage solves Equation (142) along the wall-normal direction. For a physical cell (i, j), the corresponding equation is

$$\boxed{a^y_{S,j}\, \delta T_{i,j-1} + a^y_P\, \delta T_{ij} + a^y_{N,j}\, \delta T_{i,j+1} = \delta\widetilde{T}_{ij}.} \tag{155}$$

The coefficients are

$$a^y_{S,j} = -\frac{\alpha_c v_{i,j-1}\Delta t}{2\Delta y} - \frac{\Delta t}{\mathrm{Pe}\,\Delta y^2}, \tag{156}$$

$$a^y_P = 1 + \frac{2\Delta t}{\mathrm{Pe}\,\Delta y^2}, \tag{157}$$

$$a^y_{N,j} = \frac{\alpha_c v_{i,j+1}\Delta t}{2\Delta y} - \frac{\Delta t}{\mathrm{Pe}\,\Delta y^2}. \tag{158}$$

For each fixed streamwise index i, the unknown corrections are arranged in a vector containing the physical cells and the two wall-normal ghost cells. The right-hand side is obtained from the corresponding column of the intermediate correction field, with zero right-hand-side entries for the boundary-condition rows.

The wall-normal coefficient matrix is assembled using `build_y_matrix`.

Algorithm 8 Construction of the Wall-Normal Coefficient Matrix

Require: Mesh dimensions n_x, n_y, streamwise index i, time step Δt, Peclet number Pe, velocity field v, domain length L_y, and convection coefficient α_c

Ensure: Wall-normal matrix $\mathbf{A}_y^{(i)}$

$\Delta y \leftarrow L_y/n_y$

Initialize $\mathbf{A}_y^{(i)}$ as a zero matrix of size $(n_y+2)\times(n_y+2)$

Set the bottom-wall row: $A^y_{0,0} \leftarrow 1$, $A^y_{0,1} \leftarrow 1$

Set the top-wall row: $A^y_{n_y+1,n_y} \leftarrow 1$, $A^y_{n_y+1,n_y+1} \leftarrow 1$

for $j = 1, \ldots, n_y$ **do**

$$A^y_{j,j} \leftarrow 1 + \frac{2\Delta t}{\mathrm{Pe}\,\Delta y^2}$$

$$A^y_{j,j+1} \leftarrow \frac{\alpha_c \Delta t\, v_{i,j+1}}{2\Delta y} - \frac{\Delta t}{\mathrm{Pe}\,\Delta y^2}$$

$$A^y_{j,j-1} \leftarrow -\frac{\alpha_c \Delta t\, v_{i,j-1}}{2\Delta y} - \frac{\Delta t}{\mathrm{Pe}\,\Delta y^2}$$

end for

return $\mathbf{A}_y^{(i)}$

For each physical streamwise index, the second stage is written in matrix form as

$$\boxed{\mathbf{A}_y^{(i)} \delta\mathbf{T}^{(i)} = \delta\widetilde{\mathbf{T}}^{(i)}.} \tag{159}$$

This system is also tridiagonal and is solved using the Thomas algorithm. Repeating the procedure over all physical streamwise indices yields the final temperature correction field.

In the present problem, the prescribed velocity field has $v = 0$. Consequently, the wall-normal convective terms in the residual and wall-normal matrix coefficients vanish, although the wall-normal diffusion terms remain.

6.5.6 APF Iteration and Steady-State Convergence

The complete APF procedure consists of repeatedly evaluating the spatial residual, solving the streamwise and wall-normal systems, updating the temperature field, and checking the steady-state convergence criterion.

The RMS residual is defined over physical cells as

$$\boxed{R_{\mathrm{RMS}} = \sqrt{\frac{1}{n_x n_y}\sum_{i=1}^{n_x}\sum_{j=1}^{n_y}\left[\mathcal{R}_{ij}(T)\right]^2}.} \tag{160}$$

It is evaluated using the same spatial residual as Equation (139). The iteration is terminated when

$$\boxed{R_{\mathrm{RMS}} < 10^{-12}.} \tag{161}$$

Algorithm 9 APF Pseudo-Time Marching to Steady State

Require: Initial temperature field T^0, velocity fields u, v, mesh dimensions n_x, n_y, time step Δt, parameters Pe and Br, domain dimensions L_x, L_y, convection coefficient α_c, tolerance $\varepsilon = 10^{-12}$, maximum iterations $N_{\max}$, and snapshot interval N_s

Ensure: Final temperature field T, iteration and residual histories, computation time, and temperature snapshots

$T \leftarrow T^0$
Initialize iteration and residual histories as empty
Store T^0 as the initial snapshot at iteration 0
Start the computation timer
converged $\leftarrow$ false
for $n = 1, \ldots, N_{\max}$ **do**
    Apply the temperature boundary conditions
    Initialize the intermediate correction field $\delta\widetilde{T}$ and final correction field δT

    First stage: streamwise solve
    for $j = 1, \ldots, n_y$ **do**
        Construct $\mathbf{r}^{(j)}$ using Algorithm 6
        Construct $\mathbf{A}_x^{(j)}$ using Algorithm 7
        Set the boundary entries of $\Delta t\, \mathbf{r}^{(j)}$ to zero
        Solve $\mathbf{A}_x^{(j)} \delta\widetilde{\mathbf{T}}^{(j)} = \Delta t\, \mathbf{r}^{(j)}$ using the Thomas algorithm, as given in Equation (154)
        Store the solution in the corresponding row of $\delta\widetilde{T}$
    end for

    Second stage: wall-normal solve
    for $i = 1, \ldots, n_x$ **do**
        Construct $\mathbf{A}_y^{(i)}$ using Algorithm 8
        Extract the corresponding column of $\delta\widetilde{T}$
        Solve $\mathbf{A}_y^{(i)} \delta\mathbf{T}^{(i)} = \delta\widetilde{\mathbf{T}}^{(i)}$ using the Thomas algorithm, as given in Equation (159)
        Store the solution in the corresponding column of δT
    end for

    Update the temperature using Equation (143)
    Apply the temperature boundary conditions
    Evaluate R_{RMS} using Equation (160)
    Append n to the iteration history
    Append R_{RMS} to the residual history
    if $n \bmod N_s = 0$ **then**
        Store the current temperature field as a snapshot
        Record snapshot iteration n
    end if
    if $R_{\text{RMS}} \leq \varepsilon$ **then**
        converged $\leftarrow$ true
        break
    end if
end for
Stop the timer and calculate computation time
if the final iteration is not already stored as a snapshot **then**
    Store the final temperature field and iteration
end if
if not converged **then**
    Issue a warning that the tolerance was not reached within $N_{\max}$ iterations
end if
return T, iteration history, residual history, computation time, temperature snapshots, and snapshot iterations

The APF method therefore advances the temperature field through two sets of tridiagonal solves at each pseudo-time iteration. The RMS residual measures convergence toward a solution of the discrete steady energy equation. A small residual indicates that the discrete equation is nearly satisfied, but does not by itself establish spatial accuracy or grid independence.

6.6 Explicit Runge–Kutta Implementation

The explicit Runge–Kutta methods are implemented using a common solver function, `_solve_with_explicit_r`. The method is specified through the coefficients `rk_a` and `rk_b`, which determine the intermediate stages and the final weighted update, respectively. This allows the same implementation to be used for both the second-order explicit midpoint method and the classical fourth-order Runge–Kutta method derived in the previous section.

6.6.1 Stage Evaluation and Temperature Update

At each pseudo-time iteration, the solver evaluates the right-hand-side operator defined in Equation (67) at several intermediate temperature fields. For a method with m stages, the intermediate fields and stage evaluations are expressed generally as

$$T^{(s)} = T^n + \Delta t \sum_{r=1}^{s-1} a_{sr} k_r, \qquad s = 1, \ldots, m, \tag{162}$$

$$k_s = \mathcal{F}\left(T^{(s)}\right), \qquad s = 1, \ldots, m, \tag{163}$$

$$T^{n+1} = T^n + \Delta t \sum_{s=1}^{m} b_s k_s, \tag{164}$$

where a_{sr} and b_s are the Runge–Kutta coefficients, and $T^{(1)} = T^n$. These general expressions correspond to the specific stage equations and updates given in Equations (68)–(71) for RK2 and Equations (72)–(76) for RK4.

The implementation advances only the physical-cell temperatures explicitly. At each stage, a copy of the current temperature field is created, and the intermediate stage contributions are added to its physical cells. The temperature boundary conditions are then reapplied before evaluating the spatial right-hand side. This ensures that the ghost-cell values are consistent with the boundary conditions at every stage.

After all stages have been evaluated, their right-hand sides are combined using the coefficients b_s to update the physical-cell temperatures. The boundary conditions are reapplied once more to obtain a consistent updated temperature field.

Algorithm 10 One Explicit Runge–Kutta Time Step

Require: Temperature field T^n, time step Δt, Runge–Kutta coefficients a_{sr}, b_s, velocity fields u, v, and spatial-operator parameters
Ensure: Updated temperature field T^{n+1}
Apply the temperature boundary conditions to T^n
Initialize an empty list of stage right-hand sides
for $s = 1, \ldots, m$ **do**
    Set $T^{(s)} \leftarrow T^n$
    if $s > 1$ **then**
        Construct the intermediate stage using Equation (162) using only physical-cell values
    end if
    Apply the temperature boundary conditions to $T^{(s)}$
    Evaluate k_s using Equation (163)
    Store k_s in the stage right-hand-side list
end for
Initialize the total increment to zero
for $s = 1, \ldots, m$ **do**
    Add $b_s k_s$ to the total increment over physical cells
end for
Update the physical-cell temperatures using Equation (164)
Apply the temperature boundary conditions to T^{n+1}
return T^{n+1}

For the explicit midpoint method, the solver uses two stages, with coefficients $a_{21} = 1/2$ and $b_1 = 0$, $b_2 = 1$. For classical RK4, four stages are used, with

$$a_{21} = \frac{1}{2}, \qquad a_{32} = \frac{1}{2}, \qquad a_{43} = 1,$$
$$b_1 = \frac{1}{6}, \qquad b_2 = \frac{1}{3}, \qquad b_3 = \frac{1}{3}, \qquad b_4 = \frac{1}{6}.$$

Thus, the two methods differ only in their stage coefficients and the number of right-hand-side evaluations required per iteration.

6.6.2 Pseudo-Time Marching and Convergence

The solver repeatedly advances the temperature field by one Runge–Kutta step until the steady-state residual reaches the specified tolerance or the maximum number of iterations is exceeded. Before iteration begins, the initial temperature field is checked for compatibility with the expected array dimensions and the temperature boundary conditions are applied.

After each complete time step, the RMS residual is evaluated using Equation (132). The iteration number and residual are recorded, and temperature snapshots are stored at the specified interval. The initial field is stored at iteration zero, and the final field is included even when the final iteration does not coincide with a snapshot interval.

The convergence criterion is

$$\boxed{R_{\mathrm{RMS}} \leq \varepsilon, \qquad \varepsilon = 10^{-12}.} \tag{165}$$

The computation time is measured over the pseudo-time iteration loop. If the residual tolerance is not reached within the prescribed maximum number of iterations, a warning is issued and the

final computed solution and diagnostic histories are returned.

Algorithm 11 Runge–Kutta Pseudo-Time Marching to Steady State

Require: Initial temperature T^0, time step Δt, Runge–Kutta coefficients a_{sr}, b_s, residual tolerance ε, maximum iterations $N_{\max}$, and snapshot interval N_s
Ensure: Final temperature field T, iteration and residual histories, computation time, and temperature snapshots
Validate the time step, iteration limit, snapshot interval, stage coefficients, and initial field dimensions
Apply the temperature boundary conditions to T^0
Initialize iteration and residual histories as empty
Store T^0 as the initial snapshot at iteration 0
Start the computation timer
converged $\leftarrow$ false
for $n = 1, \ldots, N_{\max}$ **do**
    Advance T by one Runge–Kutta step using Algorithm 10
    Evaluate R_{RMS} using Equation (132)
    Append n to the iteration history
    Append R_{RMS} to the residual history
    if $n \bmod N_s = 0$ **then**
        Store the current temperature field as a snapshot
        Record snapshot iteration n
    end if
    if $R_{\mathrm{RMS}} \leq \varepsilon$ **then**
        converged $\leftarrow$ true
        break
    end if
end for
Stop the timer and calculate computation time
if the final iteration is not already stored as a snapshot **then**
    Store the final temperature field and iteration
end if
if not converged **then**
    Issue a warning that the residual tolerance was not reached within $N_{\max}$ iterations
end if
return T, iteration history, residual history, computation time, temperature snapshots, and snapshot iterations

The RK2 and RK4 implementations therefore share the same boundary-condition treatment, residual evaluation, convergence criterion, and snapshot procedure. Their main differences are the number of stages and the associated Runge–Kutta coefficients. Both methods are explicit and conditionally stable; consequently, the pseudo-time step must be selected to maintain numerical stability while allowing efficient convergence to the steady solution.

6.7 Direct Sparse Solver Implementation

The direct solver is implemented through three main procedures: assembly of the sparse spatial matrix, construction of the forcing vector, and implicit pseudo-time integration. Unlike the APF and Runge–Kutta implementations, which retain ghost cells in the temperature array during the solution process, the direct solver eliminates ghost-cell temperatures from the linear-system

unknowns by substituting the boundary relations directly into the discretized equations.

6.7.1 Sparse Spatial Matrix Assembly

The function `build_spatial_matrix` constructs the sparse matrix $\mathbf{A}$ in Equation (79). The matrix is initially stored in the list-of-lists (LIL) format, which facilitates incremental insertion of its coefficients, and is converted to compressed sparse row (CSR) format after assembly.

The grid spacings and diffusion coefficients are calculated as

$$\Delta x = \frac{L_x}{n_x}, \qquad \Delta y = \frac{L_y}{n_y}, \qquad D_x = \frac{1}{\mathrm{Pe}\,\Delta x^2}, \qquad D_y = \frac{1}{\mathrm{Pe}\,\Delta y^2}. \tag{166}$$

For each physical cell (i, j), the spatial operator can be written in the five-point stencil form

$$\begin{aligned}\mathcal{L}_{ij}(T) = a_P^{(0)} T_{ij} + a_E T_{i+1,j} + a_W T_{i-1,j} \\ + a_N T_{i,j+1} + a_S T_{i,j-1} + S_{ij},\end{aligned} \tag{167}$$

where $a_P^{(0)}$ is the initial diagonal coefficient, a_E, a_W, a_N, a_S are the coefficients of the neighbouring cells, and S_{ij} is the viscous-dissipation source term. The coefficients are obtained from the central-difference discretization:

$$a_E = D_x - \frac{\alpha_c u_{i+1,j}}{2\Delta x}, \tag{168}$$

$$a_W = D_x + \frac{\alpha_c u_{i-1,j}}{2\Delta x}, \tag{169}$$

$$a_N = D_y - \frac{\alpha_c v_{i,j+1}}{2\Delta y}, \tag{170}$$

$$a_S = D_y + \frac{\alpha_c v_{i,j-1}}{2\Delta y}, \tag{171}$$

$$a_P^{(0)} = -2(D_x + D_y). \tag{172}$$

Here, α_c is the convection coefficient, which defaults to unity. The source term is independent of temperature because the velocity field is prescribed; it is therefore assembled into the forcing vector rather than the matrix $\mathbf{A}$.

For an interior cell, the coefficients of neighbouring physical-cell temperatures are inserted into the corresponding matrix entries. For example, a_E multiplies the temperature of the cell immediately to the east, $T_{i+1,j}$, so it is placed in the matrix column corresponding to that cell. The same procedure applies to the west, north, and south neighbours.

At a boundary, however, the neighbouring cell may be a ghost cell whose temperature is determined by the boundary condition rather than being an independent unknown. Its contribution must therefore be substituted into the discrete equation before the matrix is assembled. This substitution can modify the diagonal coefficient and introduce a constant contribution to the forcing vector.

Elimination of ghost-cell contributions. At the outlet, the zero-gradient condition gives

$$T_{\mathrm{GC}} = T_{\mathrm{BC}}, \tag{173}$$

where T_{BC} denotes the adjacent physical-cell temperature. Therefore, the east-neighbour contribution from Equation (167) becomes

$$a_E T_{\mathrm{GC}} = a_E T_{\mathrm{BC}}. \tag{174}$$

The ghost-cell temperature is replaced by the same physical temperature already multiplying the diagonal coefficient. Consequently, a_E is added to the diagonal:

$$A_{k,k} \leftarrow A_{k,k} + a_E. \tag{175}$$

This boundary condition introduces no additional constant term into the forcing vector.

At a prescribed-temperature boundary, the ghost-cell relation is

$$T_{\mathrm{GC}} = 2T_B - T_{\mathrm{BC}}, \tag{176}$$

where T_B is the prescribed boundary temperature. Substituting this relation into a neighbouring-cell contribution gives

$$\boxed{a_{\mathrm{BC}} T_{\mathrm{GC}} = -a_{\mathrm{BC}} T_{\mathrm{BC}} + 2a_{\mathrm{BC}} T_B.} \tag{177}$$

The first term on the right-hand side is proportional to the adjacent physical-cell temperature. Its coefficient is therefore added to the diagonal with the opposite sign:

$$A_{k,k} \leftarrow A_{k,k} - a_{\mathrm{BC}}. \tag{178}$$

The second term is independent of the unknown temperatures and is incorporated into the forcing vector.

For example, at the inlet,

$$a_W T_{0,j} = -a_W T_{1,j} + 2a_W T_{\mathrm{in}}(y_j), \tag{179}$$

so the west-neighbour coefficient is subtracted from the diagonal, while the prescribed inlet-temperature contribution is included in the forcing vector. The same procedure is applied at the bottom and top walls using their prescribed temperatures.

Consequently, the diagonal coefficient of a boundary cell is not simply the initial value $a_P^{(0)}$. It is modified by the contributions of any neighbouring ghost cells that are eliminated through the boundary relations. At a corner cell, the modifications associated with both adjoining boundaries are applied. These substitutions preserve the discretized governing equation while restricting the unknown vector to the physical-cell temperatures.

The two-dimensional cell indices are mapped to a one-dimensional index according to Equation (80):

$$k(i,j) = in_y + j, \qquad 0 \le i < n_x, \quad 0 \le j < n_y. \tag{180}$$

This ordering corresponds to flattening the physical temperature array using `ravel(order="C")`, with the wall-normal index varying fastest.

The matrix assembly procedure is summarized in Algorithm 12.

Algorithm 12 Assembly of the Sparse Spatial Matrix

Require: Mesh dimensions n_x, n_y, domain lengths L_x, L_y, Peclet number Pe, velocity fields u, v, and convection coefficient α_c
Ensure: Sparse spatial matrix $\mathbf{A}$

Calculate $\Delta x, \Delta y, D_x, D_y$ using Equation (166)
Initialize $\mathbf{A}$ as an $n_x n_y \times n_x n_y$ zero matrix in LIL format
for $i = 0, \ldots, n_x - 1$ **do**
    for $j = 0, \ldots, n_y - 1$ **do**
        Map to velocity-array indices: $I \leftarrow i + 1$, $J \leftarrow j + 1$
        $k \leftarrow i n_y + j$
        Calculate a_E, a_W, a_N, a_S using Equations (168) –(171)
        $A_{k,k} \leftarrow a_P^{(0)}$
        if $i < n_x - 1$ **then**
            $A_{k,k+n_y} \leftarrow A_{k,k+n_y} + a_E$
        else
            $A_{k,k} \leftarrow A_{k,k} + a_E$ ▷ Zero-gradient outlet
        end if
        if $i > 0$ **then**
            $A_{k,k-n_y} \leftarrow A_{k,k-n_y} + a_W$
        else
            $A_{k,k} \leftarrow A_{k,k} - a_W$ ▷ Prescribed inlet temperature
        end if
        if $j < n_y - 1$ **then**
            $A_{k,k+1} \leftarrow A_{k,k+1} + a_N$
        else
            $A_{k,k} \leftarrow A_{k,k} - a_N$ ▷ Top wall
        end if
        if $j > 0$ **then**
            $A_{k,k-1} \leftarrow A_{k,k-1} + a_S$
        else
            $A_{k,k} \leftarrow A_{k,k} - a_S$ ▷ Bottom wall
        end if
    end for
end for
Convert $\mathbf{A}$ to CSR format
return A

6.7.2 Construction of the Forcing Vector

The function `build_forcing_field` constructs the temperature-independent vector $\mathbf{b}$ in Equation (79). Rather than assembling each boundary and source contribution separately, it evaluates the existing spatial operator with all physical-cell temperatures set to zero while retaining the prescribed boundary conditions.

Accordingly, the forcing vector is defined as

$$\boxed{\mathbf{b} = \mathcal{L}(\mathbf{0}).} \tag{181}$$

A zero-valued temperature array, including ghost cells, is first initialized. The temperature boundary conditions are then applied, giving the appropriate ghost-cell values for the zero

physical-cell field. The function evaluates the right-hand side using `build_rhs` for each physical wall-normal index and copies the residual values at the physical streamwise cells into the corresponding column of the forcing field.

The field is finally flattened in C order to match the ordering used by the spatial matrix. The resulting vector contains the constant contributions from prescribed boundary temperatures and the viscous-dissipation source. The outlet condition adds no independent constant contribution because its ghost-cell temperature equals the adjacent physical-cell temperature.

Algorithm 13 Construction of the Forcing Vector

Require: Mesh dimensions n_x, n_y, inlet-temperature profile, parameters Pe, Br, velocity fields u, v, domain dimensions L_x, L_y, and convection coefficient α_c
Ensure: Forcing vector $\mathbf{b}$
1: Initialize a zero temperature array including ghost cells
2: Apply the temperature boundary conditions
3: Initialize an $n_x \times n_y$ forcing field $\mathbf{B}$
4: **for** $j = 1, \ldots, n_y$ **do**
5: Evaluate the discrete right-hand side using `build_rhs` at wall-normal index j
6: Copy its physical streamwise entries into column j of $\mathbf{B}$
7: **end for**
8: Flatten $\mathbf{B}$ in C order to obtain $\mathbf{b}$
9: **return** $\mathbf{b}$

Combining the spatial matrix and forcing vector gives

$$\frac{\mathrm{d}\mathbf{T}}{\mathrm{d}t} = \mathbf{A}\mathbf{T} + \mathbf{b}. \tag{182}$$

Because the velocity field is fixed and the boundary relations are linear, the discretized spatial operator is affine in the physical-cell temperatures and can be represented by this matrix-vector form.

6.7.3 Implicit Euler Integration and Sparse LU Factorization

The function `solve_with_direct` advances the semi-discrete system using implicit Euler pseudo-time integration. As derived in Equation (82), the temperature update satisfies

$$\boxed{(\mathbf{I} - \Delta t\, \mathbf{A})\, \mathbf{T}^{n+1} = \mathbf{T}^{n} + \Delta t\, \mathbf{b}.} \tag{183}$$

Defining

$$\mathbf{M} = \mathbf{I} - \Delta t\, \mathbf{A}, \tag{184}$$

the update is obtained by solving

$$\mathbf{M}\mathbf{T}^{n+1} = \mathbf{T}^{n} + \Delta t\, \mathbf{b}. \tag{185}$$

After assembling $\mathbf{A}$ and $\mathbf{b}$, the solver converts $\mathbf{M}$ to compressed sparse column (CSC) format and factorizes it once using the sparse LU routine `splu`. The factorization is reused throughout the pseudo-time iteration because the matrix remains unchanged when the grid, time step, velocity field, and boundary conditions are fixed.

At each iteration, the current physical-cell temperature array is flattened in C order and used to form the right-hand side $\mathbf{T}^n + \Delta t\,\mathbf{b}$. The factorized matrix object's `solve` method returns the updated temperature vector directly, without requiring the matrix to be factorized again. The new vector is reshaped to an $n_x \times n_y$ array in C order and inserted into the physical portion of the full temperature field. The ghost-cell values are subsequently reconstructed by applying the temperature boundary conditions.

This procedure is repeated until the RMS residual reaches the prescribed tolerance or the maximum iteration count is reached. At every iteration, the solver checks that the updated physical-cell temperatures are finite, evaluates the residual using Equation (132), and records the iteration number and residual. Temperature snapshots are saved at the specified interval, at convergence, or at the final permitted iteration. Computation time includes matrix construction, factorization, and pseudo-time marching.

Algorithm 14 Direct Sparse Solver with Reusable LU Factorization

Require: Initial temperature T^0, time step Δt, mesh dimensions n_x, n_y, boundary conditions, parameters Pe, Br, residual tolerance ε, maximum iterations $N_{\max}$, and snapshot interval N_s
Ensure: Final temperature field, iteration and residual histories, computation time, and temperature snapshots

Validate the time step, iteration limit, snapshot interval, and initial temperature dimensions
Apply the temperature boundary conditions to T^0
Construct $\mathbf{A}$ using Algorithm 12
Construct $\mathbf{b}$ using Algorithm 13
Form $\mathbf{M} = \mathbf{I} - \Delta t\,\mathbf{A}$ using Equation (184)
Convert $\mathbf{M}$ to CSC format and factorize it once using sparse LU factorization
Extract the physical-cell temperatures into $\mathbf{T}^0$ using C-order flattening
Initialize iteration and residual histories
Store T^0 as the initial snapshot at iteration 0
Start the computation timer
for $n = 1, \ldots, N_{\max}$ **do**
    $\mathbf{q} \leftarrow \mathbf{T}^{n-1} + \Delta t\,\mathbf{b}$
    Solve $\mathbf{M}\mathbf{T}^n = \mathbf{q}$ using the previously factorized matrix
    Check that all components of $\mathbf{T}^n$ are finite; raise an error otherwise
    Reshape $\mathbf{T}^n$ in C order and insert it into the physical cells of the temperature array
    Apply the temperature boundary conditions
    Evaluate R_{RMS} using Equation (132)
    Record the iteration number and residual
    if $n \bmod N_s = 0$ or $n = N_{\max}$ or ($\varepsilon > 0$ and $R_{\mathrm{RMS}} \le \varepsilon$) **then**
        Store the current temperature field as a snapshot
        Record snapshot iteration n
    end if
    if $\varepsilon > 0$ and $R_{\mathrm{RMS}} \le \varepsilon$ **then**
        break
    end if
end for
Stop the timer and calculate computation time
return final temperature field, histories, computation time, and snapshots

The direct solver therefore combines the sparse representation of the spatial operator with implicit Euler pseudo-time marching and a reusable sparse LU factorization. Eliminating ghost-cell

unknowns keeps the linear system restricted to the physical temperatures, while incorporating their effects through modified boundary-cell coefficients and the forcing vector. Reusing the factorization avoids repeating matrix construction and factorization at every iteration, although each time step still requires a sparse triangular solve.

6.7.4 Verification of the Direct Solver

The direct solver implementation was examined through several verification tests. First, the assembled sparse matrix and forcing vector were checked against the original spatial operator by evaluating the residual independently using `build_rhs` and comparing it with $\mathbf{AT} + \mathbf{b}$ for an arbitrary temperature field. Second, the accuracy of the implicit Euler linear-system solution was verified by evaluating the algebraic residual $\mathbf{M}\mathbf{T}^{n+1} - (\mathbf{T}^n + \Delta t\, \mathbf{b})$. Third, the converged temperature fields obtained using the direct and APF methods were compared on the same grid and under identical initial and boundary conditions, using the relative L_2 difference and maximum absolute difference. Additionally, a time-step test was implemented to examine the convergence behaviour, residual evolution, and computational cost of the direct solver for different pseudo-time steps.

7. Results

7.1 Grid Convergence Without an Exact Solution

Since an exact solution for the complete temperature field is unavailable, spatial convergence is assessed by comparing solutions on three successively refined grids. Each solution must satisfy the steady-state residual tolerance before the grid-to-grid differences are evaluated.

The grid levels used are

$$M_1 = (25, 10), \qquad M_2 = (50, 20), \qquad M_3 = (100, 40), \tag{186}$$

where each pair denotes the number of cells in the streamwise and wall-normal directions. Both grid dimensions are doubled at each refinement, giving a refinement ratio of two.

7.1.1 Successive-Grid Differences

Because the solutions are cell-centred and defined on different grids, each fine-grid solution is restricted to its corresponding coarser grid by averaging each 2×2 block:

$$T_{ij}^{f\to c} = \frac{1}{4} \sum_{r=0}^{1} \sum_{s=0}^{1} T_{2i+r,\,2j+s}^{f}, \tag{187}$$

where the indices refer to the physical-cell arrays using zero-based indexing.

The RMS difference between successive solutions is then

$$D_{k,k+1} = \sqrt{\frac{1}{n_x n_y} \sum_{i=1}^{n_x} \sum_{j=1}^{n_y} \left(T_{ij}^{M_k} - T_{ij}^{M_{k+1}\to M_k} \right)^2}. \tag{188}$$

Algorithm 15 APF Grid Self-Convergence Analysis

Require: Grid levels $M_1 = (25, 10)$, $M_2 = (50, 20)$, $M_3 = (100, 40)$; residual tolerance ε; pseudo-time step Δt

Ensure: Successive-grid differences D_{12}, D_{23}, observed order, fitted order, and convergence diagnostics

Initialize solution, residual, iteration, and computation-time records
for each grid M_k, $k = 1, 2, 3$ **do**
    Initialize temperature and velocity fields
    Solve using APF with time step Δt
    Evaluate the final residual using Equation (132)
    if $R_{\mathrm{RMS}} > \varepsilon$ **then**
        Stop the analysis and report non-convergence
    end if
    Store the converged temperature field, residual, iteration count, and computation time
end for
for $k = 1, 2$ **do**
    Extract physical-cell temperatures from M_k and M_{k+1}
    Restrict the fine-grid solution to the coarser grid using Equation (187)
    Calculate $D_{k,k+1}$ using Equation (188)
end for
if $D_{12} > 0$ and $D_{23} > 0$ **then**
    Calculate the observed order:

$$p_{\mathrm{obs}} = \frac{\ln(D_{12}/D_{23})}{\ln 2}$$

else
    Set the observed order to undefined
end if
Calculate the characteristic spacings $h_k = \sqrt{\Delta x_k \Delta y_k}$ for $k = 1, 2$
Fit $\ln D_{k,k+1}$ against $\ln h_k$ using least squares and record the fitted slope p_{fit}
Plot the successive-grid differences against characteristic spacing on logarithmic axes
Save the differences, observed and fitted orders, residuals, iteration counts, computation times, and plot
return convergence results and diagnostics

7.1.2 Observed Spatial Convergence Order

Assuming that the solutions are in the asymptotic convergence range, their successive-grid differences approximately satisfy

$$\frac{D_{23}}{D_{12}} \approx \frac{1}{2^p}. \tag{189}$$

The observed spatial order is therefore estimated as

$$\boxed{p_{\mathrm{obs}} = \frac{\ln(D_{12}/D_{23})}{\ln 2}.} \tag{190}$$

The fitted order p_{fit} is obtained from a least-squares linear fit of $\ln D_{k,k+1}$ against $\ln h_k$. For the three grid levels used here, this fit contains only two successive-grid differences, so its slope equals the pairwise observed-order estimate, apart from numerical round-off.

These estimates indicate the rate at which successive-grid differences decrease under refinement; they do not directly measure the absolute error relative to an exact solution. Their reliability depends on adequate steady-state convergence and consistent spatial convergence across the grids.

7.1.3 Grid-Convergence Results for All Solvers

The spatial self-convergence study was performed using the APF, RK2, RK4, and direct implicit Euler methods. All methods were tested with $\mathrm{Pe} = 17.5$, $\mathrm{Br} = 0.2$, and the linear inlet condition $T_{\mathrm{in}}(y) = y$ on three successively refined grids: 25×10, 50×20, and 100×40. The pseudo-time step was selected separately for each method. All calculations were advanced until the steady-state RMS residual fell below 10^{-12}.

The resulting residuals, successive-grid differences, and observed convergence orders are summarized in Table 1.

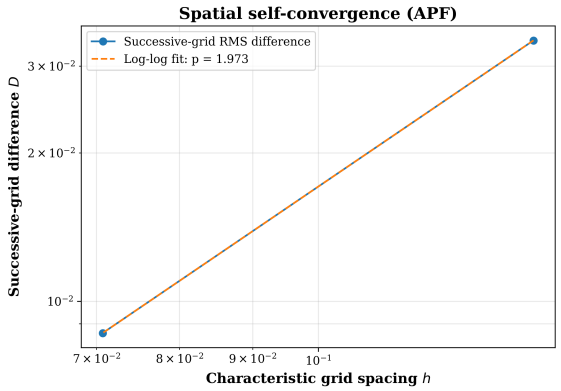
Table 1: Steady-state residuals and spatial self-convergence results for the four numerical methods.

Method	Δt	$R_{25\times10}$	$R_{50\times20}$	$R_{100\times40}$	D_{12}	D_{23}	p_{obs}
APF	0.100	6.9352×10^{-13}	9.4180×10^{-13}	9.1306×10^{-13}	3.3837×10^{-2}	8.6219×10^{-3}	1.9725
RK2	0.003	9.8784×10^{-13}	9.6466×10^{-13}	9.8259×10^{-13}	3.3837×10^{-2}	8.6219×10^{-3}	1.9725
RK4	0.001	9.9859×10^{-13}	9.9082×10^{-13}	9.8579×10^{-13}	3.3837×10^{-2}	8.6219×10^{-3}	1.9725
Direct	0.100	7.3997×10^{-13}	5.8316×10^{-13}	6.7761×10^{-13}	3.3837×10^{-2}	8.6219×10^{-3}	1.9725

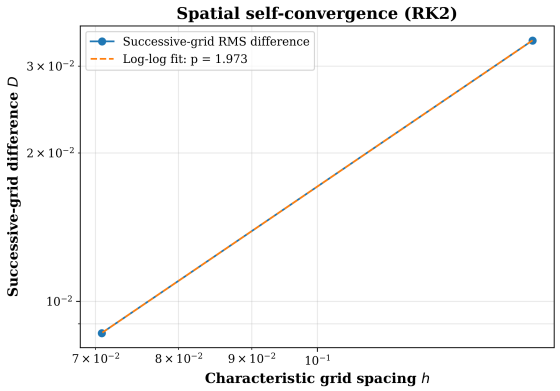
All four methods satisfy the prescribed steady-state residual tolerance on every grid. Moreover, the successive-grid RMS differences are identical across the four methods to the reported numerical precision. The observed spatial convergence order, calculated using Equation (190), is $p_{\mathrm{obs}} = 1.9725$ in every case.

The agreement between the methods is consistent with their convergence toward the same discrete steady-state solution. Although their pseudo-time integration procedures and time steps differ, they employ the same spatial discretization and therefore approach the same steady-state equation. The different pseudo-time steps affect the convergence process rather than the limiting spatial solution.

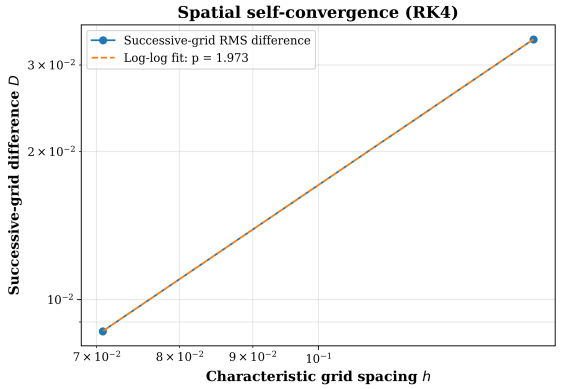
Figure 1 presents the successive-grid differences against the characteristic grid spacing for all four methods. The log–log plots exhibit approximately second-order behaviour, consistent with the expected accuracy of the central-difference spatial discretization.



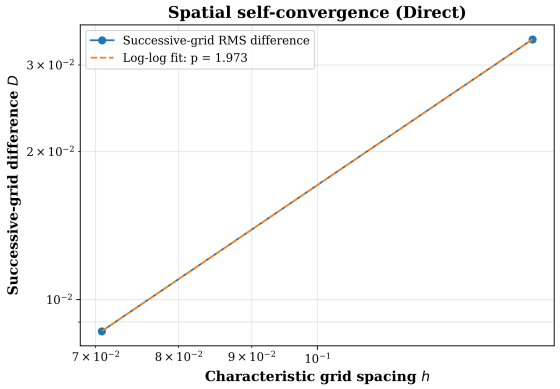
(a) APF



(b) Explicit midpoint RK2



(c) Classical RK4



(d) Direct implicit Euler

Figure 1: Spatial self-convergence of the four numerical methods for the linear inlet condition. Successive-grid RMS differences are plotted against characteristic grid spacing on logarithmic axes. All methods yield an observed spatial convergence order of approximately two.

Overall, the results demonstrate consistent second-order spatial self-convergence across all four numerical methods. The agreement in successive-grid differences also supports the consistency of their implementations and their convergence toward a common discrete steady-state solution. However, self-convergence does not independently establish the absolute error of the finest-grid solution.

7.2 Temporal Accuracy Assessment

Temporal accuracy is assessed through a self-convergence study in which the spatial grid and integration horizon are kept fixed while the pseudo-time step is successively reduced. For each time step, the solver advances the initial temperature field over the same total pseudo-time interval. This allows the differences between the resulting temperature fields to be attributed primarily to temporal discretization.

The computational domain is discretized using a fixed grid of 25×10 cells. The initial time step is $\Delta t_0 = 0.05$, and three successive halvings are applied, giving four time-step levels:

$$\Delta t_\ell = \frac{\Delta t_0}{2^\ell}, \qquad \ell = 0, 1, 2, 3. \tag{191}$$

All calculations are performed over the same final pseudo-time $\tau_f = 0.5$. Consequently, the required number of iterations for each time step is

$$N_\ell = \frac{\tau_f}{\Delta t_\ell}, \tag{192}$$

which gives 10, 20, 40, and 80 iterations, respectively.

To ensure that every calculation reaches the same integration horizon, residual-based early termination is disabled by setting the residual tolerance to zero. The maximum iteration count is set equal to the required number of time steps, and the actual number of iterations is checked after each run.

7.2.1 Successive-Time-Step Differences

Since all solutions are obtained on the same spatial grid, their physical-cell temperature fields can be compared directly, without spatial interpolation. Ghost cells are excluded from the comparison.

The RMS difference between solutions obtained using two successive time steps is defined as

$$D_{\ell,\ell+1} = \sqrt{\frac{1}{n_x n_y} \sum_{i=1}^{n_x} \sum_{j=1}^{n_y} \left(T_{ij}^{\Delta t_\ell} - T_{ij}^{\Delta t_{\ell+1}} \right)^2}, \tag{193}$$

where both solutions correspond to the same final pseudo-time and spatial grid.

7.2.2 Observed Temporal Convergence Order

Assuming that the numerical solutions are in the asymptotic temporal convergence range, the successive-time-step differences approximately satisfy

$$D_{\ell,\ell+1} \propto (\Delta t_\ell)^p, \tag{194}$$

where p is the observed temporal order of convergence.

Since each time-step refinement reduces Δt by a factor of two, the observed order between successive difference estimates is

$$\boxed{p_{\mathrm{obs},\ell} = \frac{\ln \left(D_{\ell,\ell+1} / D_{\ell+1,\ell+2} \right)}{\ln 2}.} \tag{195}$$

In addition to the pairwise estimates, a fitted temporal order is obtained through a least-squares linear fit of $\ln D_{\ell,\ell+1}$ against $\ln \Delta t_\ell$:

$$\ln D_{\ell,\ell+1} = p_{\mathrm{fit}} \ln \Delta t_\ell + C, \tag{196}$$

where p_{fit} is the fitted slope and C is the intercept.

Algorithm 16 Temporal Self-Convergence Analysis

Require: Fixed mesh M, initial time step Δt_0, number of time-step levels $N_{\Delta t} = 4$, final pseudo-time τ_f, solver, and physical parameters

Ensure: Successive-time-step differences, observed and fitted temporal orders, and convergence diagnostics

Generate the time-step sequence using Equation (191)
Initialize solution and diagnostic records
for each time step Δt_ℓ **do**
    Calculate the required iteration count N_ℓ using Equation (192)
    Initialize the temperature and velocity fields on the fixed mesh
    Set residual tolerance to zero and maximum iterations to N_ℓ
    Execute the selected solver
    Verify that exactly N_ℓ iterations were performed
    Check that the final temperature field contains only finite values
    Store the final temperature field, residual, iteration count, and computation time
end for
for each pair of successive time steps **do**
    Extract physical-cell temperatures from both solutions
    Calculate the RMS difference using Equation (193)
    Store the resulting difference
end for
for each pair of successive difference estimates **do**
    Calculate the observed temporal order using Equation (195)
end for
Fit $\ln D_{\ell,\ell+1}$ against $\ln \Delta t_\ell$ using least squares
Record the fitted slope p_{fit}
Generate the log–log temporal self-convergence plot
Save time steps, differences, observed orders, fitted order, residuals, iteration counts, and computation times
return temporal convergence results and diagnostics

The temporal self-convergence study provides an estimate of the time-integration accuracy without requiring an exact transient solution. Its reliability depends on comparing solutions at the same integration horizon, using sufficiently fine time steps to reach the asymptotic convergence range.

7.2.3 Temporal Accuracy Results

The temporal accuracy of the four numerical methods was assessed on a fixed 25×10 spatial grid using $\mathrm{Pe} = 17.5$, $\mathrm{Br} = 0.2$, and a linear inlet temperature profile $T_{\text{in}}(y) = y$. All calculations were advanced over the same final pseudo-time $\tau_f = 0.5$, using successively halved time steps:

$$\Delta t = \{0.05,\ 0.025,\ 0.0125,\ 0.00625\}. \tag{197}$$

The residual-based steady-state termination criterion was disabled to ensure that all solutions were evaluated at the same integration horizon. Consequently, the calculations were performed for exactly 10, 20, 40, and 80 iterations, respectively.

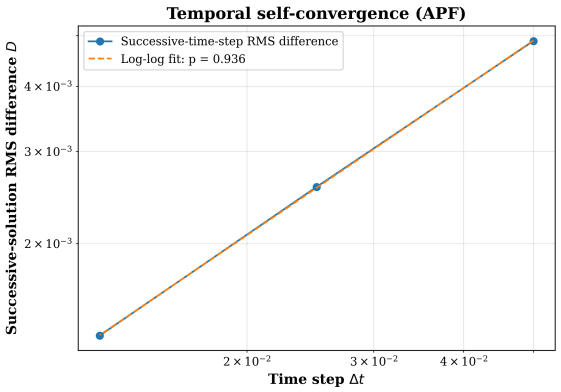
The successive-time-step RMS differences and the resulting temporal convergence orders are

summarized in Table 2.

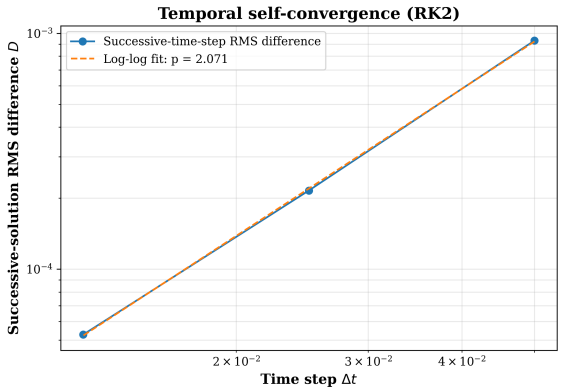
Table 2: Temporal self-convergence results for the four numerical methods.

Method	$D_{0,1}$	$D_{1,2}$	$D_{2,3}$	$p_{\mathrm{obs},1}$	$p_{\mathrm{obs},2}$	p_{fit}
APF	4.8819×10^{-3}	2.5673×10^{-3}	1.3341×10^{-3}	0.92721	0.94437	0.93579
RK2	9.3241×10^{-4}	2.1548×10^{-4}	5.2779×10^{-5}	2.11338	2.02955	2.07147
RK4	2.0929×10^{-5}	1.2419×10^{-6}	7.5599×10^{-8}	4.07489	4.03803	4.05646
Direct	4.7960×10^{-3}	2.5499×10^{-3}	1.3306×10^{-3}	0.91137	0.93840	0.92488

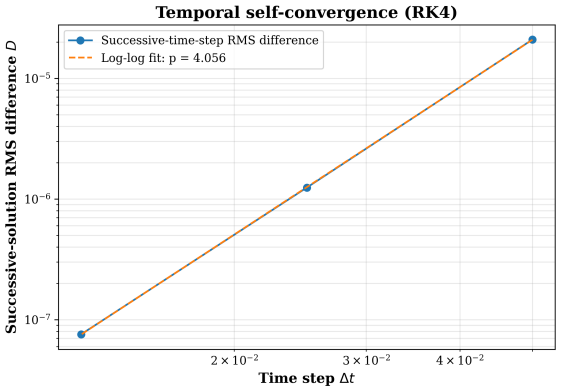
Here, $D_{0,1}$, $D_{1,2}$, and $D_{2,3}$ denote the RMS differences between solutions obtained using successive time steps. The observed orders are calculated using Equation (195), while p_{fit} represents the slope of the corresponding log–log least-squares fit.



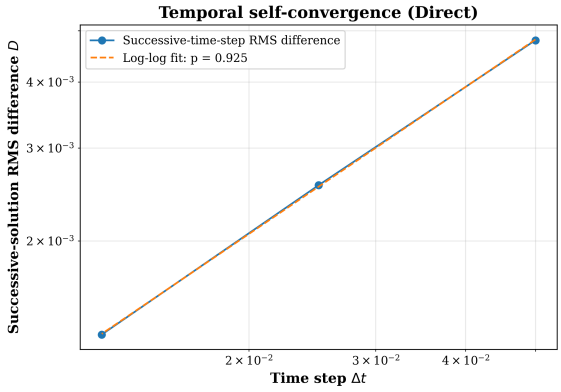
(a) Approximate factorization (APF)



(b) Explicit midpoint RK2



(c) Classical RK4



(d) Direct implicit Euler

Figure 2: Temporal self-convergence of the four numerical methods on the fixed 25×10 grid. Successive-time-step RMS differences are plotted against the time step on logarithmic axes.

The results indicate approximately first-order temporal convergence for both APF and the direct implicit Euler method, with fitted orders of 0.93579 and 0.92488, respectively. The explicit midpoint method exhibits approximately second-order convergence, with a fitted order of 2.07147. The classical fourth-order Runge–Kutta method achieves a fitted order of 4.05646. These estimates are consistent with the expected temporal accuracy of the respective integration schemes.

The successive-difference values also decrease systematically as the time step is refined. In particular, the RK4 method exhibits substantially smaller differences than the other methods for the tested time-step range, consistent with its higher temporal order. The pairwise estimates for all methods are reasonably close to their respective theoretical orders, indicating that the selected time-step range provides a consistent assessment of temporal convergence.

The final RMS residuals remain approximately between 0.391 and 0.393 for all methods. This is expected because the calculations are intentionally terminated at the fixed pseudo-time $\tau_f = 0.5$, before reaching steady state. The residual values therefore characterize the transient solution at the selected integration horizon and are not used as a convergence criterion in this study.

Overall, the results confirm the expected temporal orders of the four numerical methods: first order for APF and implicit Euler, second order for explicit midpoint RK2, and fourth order for classical RK4.

7.3 Stability and Time-Step Efficiency

7.3.1 Stability Analysis

To assess the stability of the time-integration methods, the semi-discrete system was expressed in matrix form as

$$\frac{d\mathbf{T}}{dt} = \mathbf{AT} + \mathbf{b}, \tag{198}$$

where $\mathbf{A}$ represents the linear spatial discretization, including the contributions of the boundary conditions, and $\mathbf{b}$ contains the temperature-independent source terms. Since the prescribed velocity field makes the viscous-dissipation source independent of temperature, it does not affect the eigenvalue-based stability analysis.

The eigenvalues of $\mathbf{A}$ were computed numerically for the 50×20 grid. The maximum real part and maximum eigenvalue magnitude were found to be

$$\max_k \Re(\lambda_k) = -1.216672 \times 10^1, \qquad \max_k |\lambda_k| = 1.142857 \times 10^2. \tag{199}$$

The negative real parts indicate that the semi-discrete system is asymptotically stable.

The absolute stability of a time-integration method can be examined by applying it to the scalar test equation $\dot{w} = \lambda w$. Defining $z = \lambda \Delta t$, the numerical solution takes the form $w^{n+1} = G(z)w^n$, where stability requires $|G(z)| \leq 1$. The amplification factors for the methods considered here are

$$G_{\mathrm{IE}}(z) = \frac{1}{1-z}, \tag{200}$$

$$G_{\mathrm{RK2}}(z) = 1 + z + \frac{z^2}{2}, \tag{201}$$

$$G_{\mathrm{RK4}}(z) = 1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24}, \tag{202}$$

for implicit Euler, explicit midpoint RK2, and classical explicit RK4, respectively. For a given time step, the stability condition is evaluated over the computed spectrum:

$$\max_k |G(\lambda_k \Delta t)| \leq 1. \tag{203}$$

Implicit Euler is A-stable and therefore stable for the computed spectrum, since all eigenvalues have negative real parts. In contrast, the explicit methods have conditional stability, and their allowable time steps depend on the eigenvalue distribution. The maximum stable time step for each method was estimated numerically using the search procedure presented in Algorithm 17.

Algorithm 17 Numerical estimation of the maximum stable time step

Require: Eigenvalues $\{\lambda_k\}$, integration scheme, initial time step Δt_0
Ensure: Estimated maximum stable time step
$\alpha \leftarrow \max_k \Re(\lambda_k)$
if $\alpha >$ stability tolerance **then**
  return NaN ▷ The semi-discrete system is unstable
end if
if scheme is implicit Euler **then**
  return ∞ ▷ Implicit Euler is A-stable
end if
$\Delta t_{\text{low}} \leftarrow 0$
$\Delta t_{\text{high}} \leftarrow \max(\Delta t_0, 10^{-12})$
bracketed $\leftarrow$ False
▷ Bracket the stability boundary
for $m = 1, \ldots, N_{\text{double}}$ **do**
  if $\max_k |G(\lambda_k \Delta t_{\text{high}})| > 1$ **then**
    bracketed $\leftarrow$ True
    break
  end if
  $\Delta t_{\text{low}} \leftarrow \Delta t_{\text{high}}$
  $\Delta t_{\text{high}} \leftarrow 2\Delta t_{\text{high}}$
end for
if not bracketed **then**
  return ∞ ▷ No instability found within the search range
end if
▷ Refine the boundary by bisection
for $m = 1, \ldots, N_{\text{bisect}}$ **do**
  $\Delta t_{\text{mid}} \leftarrow \frac{1}{2}(\Delta t_{\text{low}} + \Delta t_{\text{high}})$
  if $\max_k |G(\lambda_k \Delta t_{\text{mid}})| \leq 1$ **then**
    $\Delta t_{\text{low}} \leftarrow \Delta t_{\text{mid}}$
  else
    $\Delta t_{\text{high}} \leftarrow \Delta t_{\text{mid}}$
  end if
end for
return Δt_{low} ▷ Return the stable side of the estimated limit

At the configured time step $\Delta t = 0.1$, the maximum amplification factors were 0.45113, 54.87755, and 516.90726 for implicit Euler, RK2, and RK4, respectively. Thus, implicit Euler was stable at this time step, whereas both explicit methods were unstable. The numerical search estimated maximum stable time steps of 0.01750 for RK2 and 0.0243713 for RK4. The estimated stability limit of RK4 is therefore greater than that of RK2 for the present spatial discretization.

The results are summarized in Table 3. The implicit Euler method has no finite absolute-stability limit for the computed spectrum. The AFI value represents the largest tested convergent time step, rather than a theoretically established stability bound. The eigenvalue-based stability analysis for implicit Euler should also not be interpreted as a formal stability result for AFI, which uses approximate factorization and was assessed separately through numerical convergence tests.

For the direct implicit solver, a representative time step of $\Delta t = 1$ was selected for the efficiency

Table 3: Stability limits and selected pseudo-time steps for the numerical methods.

Method	$\max_k \lvert G\rvert$ at $\Delta t = 0.1$	Maximum stable or selected Δt	Assessment
AFI	–	0.271 (largest tested)	Largest tested value for which convergence was obtained.
Direct implicit Euler	0.45113	No finite stability limit; 1.0 selected	A-stable for the computed eigenvalue spectrum.
Explicit RK2	54.87755	0.01750 (estimated)	Conditionally stable; unstable at $\Delta t = 0.1$.
Explicit RK4	516.90726	0.0243713 (estimated)	Conditionally stable; unstable at $\Delta t = 0.1$.

comparison. This choice is substantially more moderate than the very large values identified in the preliminary time-step sweep. It is nevertheless expected to provide strong modal damping. In particular, for backward Euler and the computed spectrum,

$$\max_k |G_{\mathrm{IE}}(\lambda_k \Delta t)| \leq \frac{1}{1 - \Delta t \max_k \Re(\lambda_k)}. \tag{204}$$

At $\Delta t = 1$, this gives

$$\max_k |G_{\mathrm{IE}}(\lambda_k)| \leq 0.07595. \tag{205}$$

The selected time step must still be checked through the steady-state residual to verify convergence.

These stability limits concern pseudo-time integration toward the steady solution and do not establish the accuracy of a physical transient simulation.

7.3.2 Time-Step Efficiency

A time-step efficiency study was performed for AFI and the direct implicit solver on the 50×20 grid. Several candidate time steps were tested for each method, with three independent runs per candidate. Each run started from the initial temperature field and was considered converged when the steady-state residual satisfied the prescribed tolerance of 10^{-12}. All tested cases in the preliminary study converged in all three runs.

For AFI, the fastest tested case used $\Delta t = 0.266$, with a median computation time of 5.6624 s, 296 iterations, and a final residual of 9.51×10^{-13}. Its runtime was only marginally lower than that obtained with $\Delta t = 0.265$, indicating a relatively shallow minimum in the tested range rather than a precisely established optimal time step. Figure 3 shows the computation time across the tested AFI time steps.

For the direct implicit solver, the representative value $\Delta t = 1$ was selected instead of the very large time step that produced the minimum runtime in the preliminary sweep. The preliminary sweep identified $\Delta t = 20000$ as its fastest tested case, but this extremely aggressive pseudo-time step was not retained for the comparison. The direct solver must therefore be rerun at $\Delta t = 1$ to determine its median computation time, iteration count, and final residual. These results will be used to assess its practical efficiency alongside AFI under the selected settings.

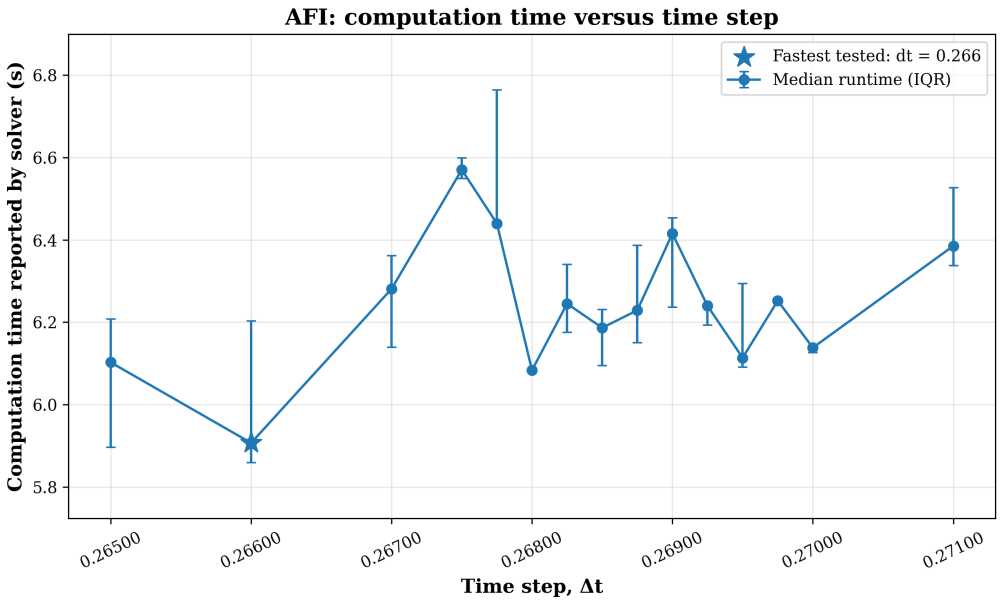


Figure 3: Median computation time of AFI as a function of the pseudo-time step. The minimum among the tested values occurs at $\Delta t = 0.266$.

7.3.3 Comparison of the Converged Solutions

To verify that the two solvers produce the same steady-state solution, their temperature fields at the selected time steps should be compared over the physical cells, excluding ghost cells. The relative L_2 difference is defined as

$$\varepsilon_{\mathrm{AFI,Direct}} = \frac{\|\mathbf{T}_{\mathrm{AFI}} - \mathbf{T}_{\mathrm{Direct}}\|_2}{\|\mathbf{T}_{\mathrm{Direct}}\|_2}. \tag{206}$$

This quantity should be recalculated using the direct solution obtained with $\Delta t = 1$. The previously obtained value of 1.15×10^{-14} corresponds to the direct-solver run at $\Delta t = 20000$ and is therefore not reported as the result of the revised comparison.

The computation-time comparison should likewise use the new direct-solver measurement at $\Delta t = 1$. Since the direct solver's runtime and iteration count can change with the selected time step, no speedup ratio is reported until the revised run has been completed. Any resulting performance comparison remains specific to the present grid, problem configuration, and implementation.

7.4 Comparison of Solver Performance

The computational performance of the four numerical methods was compared using the selected pseudo-time steps: $\Delta t = 0.265$ for AFI, $\Delta t = 1$ for the direct implicit solver, $\Delta t = 0.017$ for RK2, and $\Delta t = 0.024$ for RK4. These values were selected based on the preceding time-step and stability analyses. The explicit methods use time steps slightly below their estimated stability limits, while the direct solver uses a representative time step of 1 rather than an excessively large value.

All simulations were performed on the same 50×20 grid with $\mathrm{Pe} = 17.5$, $\mathrm{Br} = 0.2$, and

a convection coefficient of 1. Five independent runs were performed for each method, with the initial temperature field reconstructed before every run. The convergence criterion was a steady-state residual tolerance of 10^{-12}. The elapsed wall-clock time was measured around each solver call, including solver-specific setup operations such as matrix assembly and factorization. The construction of the initial temperature and velocity fields and the independent residual evaluation were excluded from the measured time.

Table 4 summarizes the results. All five runs of each method converged to the prescribed residual tolerance. The reported computation times are the medians of the five runs, and the standard deviations indicate the variation in runtime.

Table 4: Performance comparison of the four numerical methods on the 50×20 grid.

Method	Δt	Iterations	Median time (s)	Std. dev. (s)	Final residual
AFI	0.265	295	6.2817	0.1800	9.398×10^{-13}
Direct implicit Euler	1.0	16	0.1403	0.0094	6.322×10^{-13}
Explicit RK2	0.017	392	8.0148	0.1089	9.907×10^{-13}
Explicit RK4	0.024	309	10.7187	0.1100	9.748×10^{-13}

In addition to the final performance measurements, the convergence histories were examined to compare the behavior of the methods throughout the iterations. Figures 4 and 5 show the steady-state residual as a function of iteration number and elapsed computation time, respectively. For each method, the plotted history corresponds to the successful run whose total runtime was closest to the median runtime of its five runs. This provides a representative convergence history for each method without superimposing all five runs.

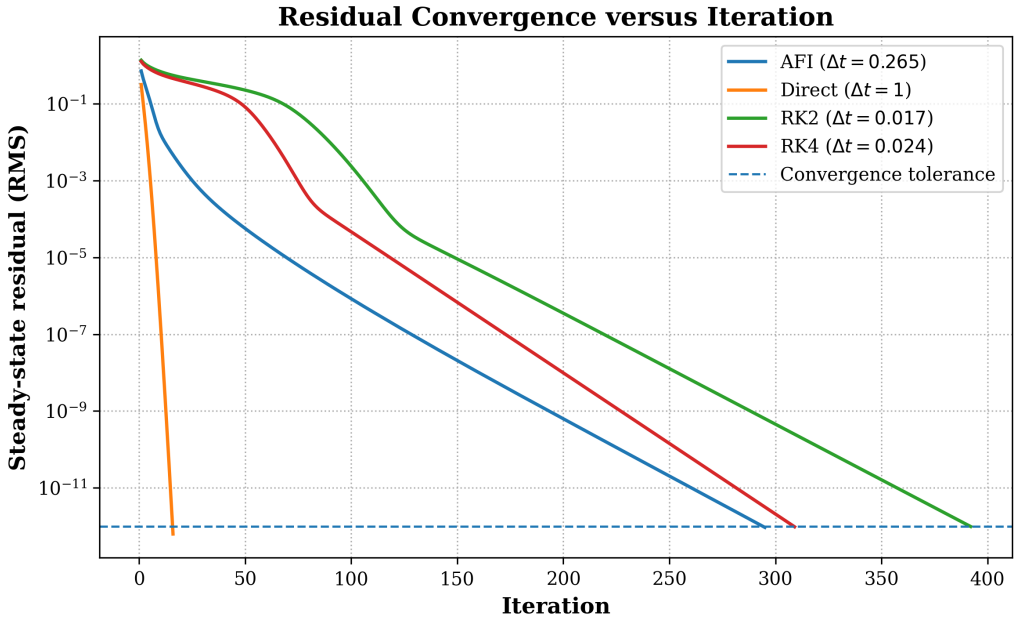


Figure 4: Steady-state residual versus iteration number for the four numerical methods. The dashed line indicates the convergence tolerance of 10^{-12}.

Figure 4 compares the convergence behavior in terms of iteration count. The direct implicit solver reached the prescribed tolerance in only 16 iterations, compared with 295 for AFI, 392 for RK2, and 309 for RK4. RK4 required fewer iterations than RK2, while the direct method converged in substantially fewer iterations than the other methods.

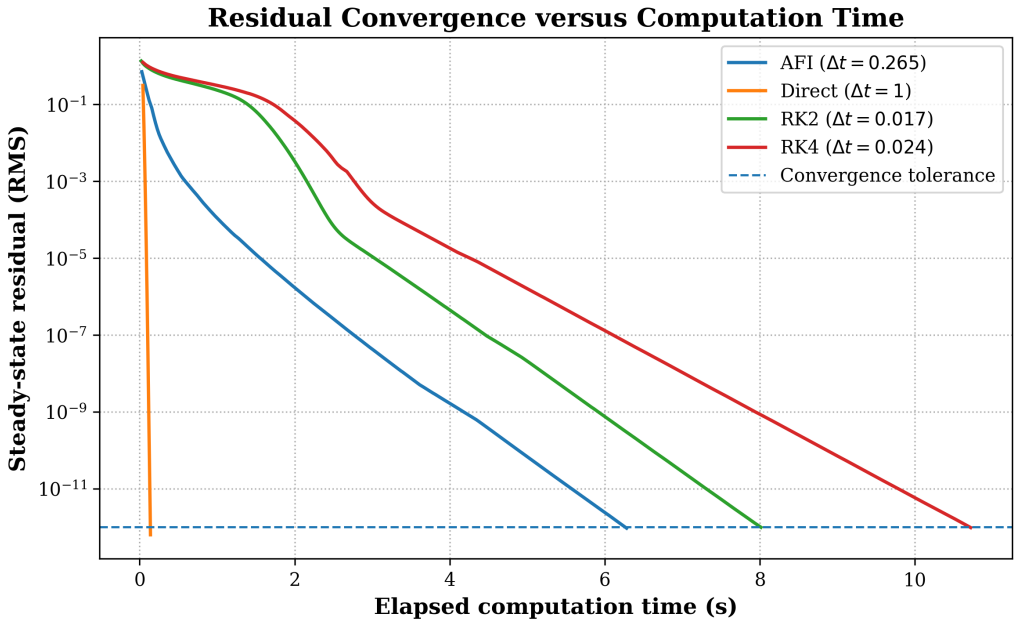


Figure 5: Steady-state residual versus elapsed computation time for the four numerical methods. The dashed line indicates the convergence tolerance of 10^{-12}.

The residual-versus-time histories provide a direct comparison of the computational effort required to reach the prescribed convergence tolerance. The direct implicit solver achieved the lowest median computation time, at 0.1403 s. Relative to this method, AFI, RK2, and RK4 required approximately 44.8, 57.1, and 76.4 times more computation time, respectively. Although RK4 required fewer iterations than RK2, its overall computation time was higher. This is consistent with the greater computational cost of its four-stage time-integration procedure compared with the two-stage RK2 method.

AFI was faster than both explicit Runge–Kutta methods for the selected time steps, despite requiring more iterations than RK4 and fewer than RK2. This indicates that iteration count alone does not determine the computational cost; the operations required within each iteration also influence the total runtime.

All four methods satisfied the same steady-state residual criterion in all five runs. The direct implicit solver was the most computationally efficient among the tested methods. However, these runtime ratios apply specifically to the present grid, implementation, and selected time steps. They should not be interpreted as general performance rankings for other grid sizes or problem configurations.

7.5 Pseudo-Time Evolution of the Temperature Field

The development of the temperature field during the numerical solution is illustrated in Figure 6. The sequence of temperature distributions shows the combined effects of convection, thermal diffusion, and viscous dissipation as the solution evolves from the initial condition toward the steady state.

The effect of convection is evident from the deformation of the temperature contours along the channel. Since the prescribed parabolic velocity profile varies across the channel height, fluid near the centreline is transported downstream faster than fluid near the walls. This produces a

curved, approximately hyperbolic pattern in the temperature distribution.

Viscous dissipation introduces a symmetric heating contribution across the channel. This follows from the squared wall-normal velocity gradient, which is symmetric about the channel centreline and reaches its maximum magnitude near the two walls. Consequently, the temperature in the fluid cells adjacent to the lower wall initially increases due to viscous heating. As the solution develops, this local temperature increase is followed by a slight decrease, as convective transport of relatively cooler fluid from the inlet competes with the heating contribution.

The effect of thermal diffusion is particularly visible in the downstream region of the channel. Heat is conducted in the wall-normal direction from the warmer upper region toward the cooler lower region, leading to a gradual redistribution and smoothing of the temperature field across the channel height.

Overall, the temperature evolution illustrates the interaction between the three transport mechanisms and their contributions to the resulting steady-state temperature distribution.

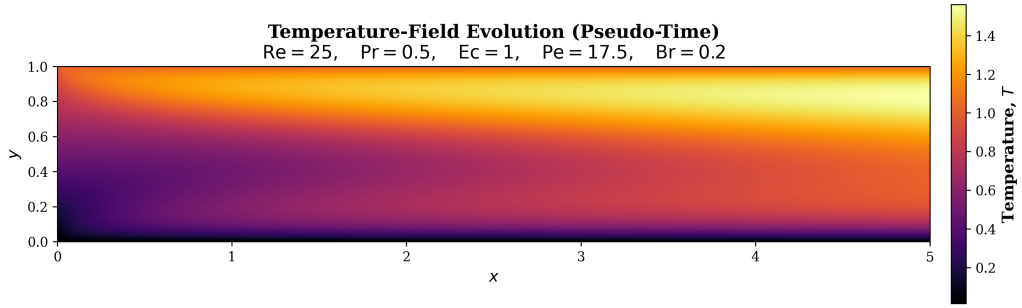


Figure 6: Representative temperature-field distribution during pseudo-time integration.

Click here to view the complete temperature evolution animation (gif).

7.6 Parameter Study

A parameter study was conducted to examine the effects of viscous dissipation, convection strength, and the Peclet number on the steady-state temperature distribution. All cases were solved on a 100×40 mesh using the APF solver, with a pseudo-time step of $\Delta t = 0.05$ and a convergence tolerance of 10^{-10}. The initial temperature and inlet temperature profiles were both prescribed as $T = y$.

7.6.1 Effect of the Brinkman Number

The Brinkman number controls the strength of viscous dissipation relative to thermal diffusion. Three cases were considered, with $Br = 0$, 0.07, and 0.2, while the Peclet number and convection coefficient were kept fixed at $Pe = 17.5$ and $\gamma_c = 1$.

For $Br = 0$, the temperature field remains close to the initial linear distribution. This behaviour is consistent with the prescribed boundary conditions: the initial and inlet profiles are both $T = y$, and the temperature distribution satisfies the wall temperatures. In the absence of viscous dissipation, this profile is also a steady solution of the energy equation.

As the Brinkman number increases, viscous heating produces a progressively larger departure from the initial linear distribution. Since the dissipation source is proportional to the square

of the velocity gradient, its contribution is strongest near the channel walls, where the velocity gradient is largest. The upper part of the channel reaches higher temperatures due to the combined effects of viscous heating and the prescribed higher temperature at the upper wall.

Figure 7 compares the resulting temperature fields for the three Brinkman numbers. All contours use the same temperature scale to facilitate comparison.

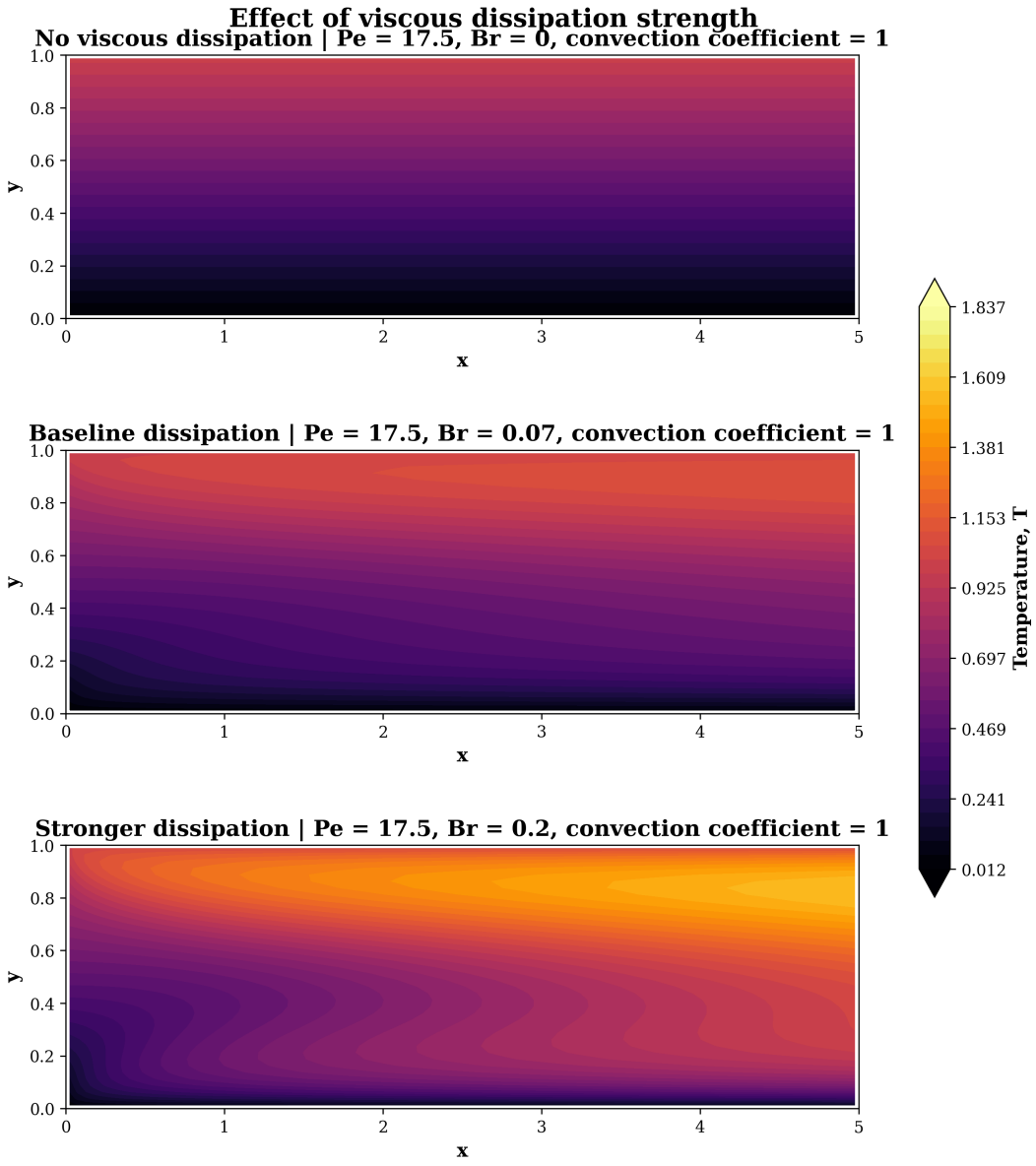


Figure 7: Effect of the Brinkman number on the steady-state temperature distribution for $Pe = 17.5$ and $\gamma_c = 1$.

7.6.2 Effect of the Convection Coefficient

The convection coefficient γ_c scales the convective contribution in the energy equation. Its effect was examined using $\gamma_c = 0.4$, 1, and 10, while keeping $Pe = 17.5$ and $Br = 0.2$ constant.

For a large convection coefficient, the temperature field remains closer to the imposed inlet profile. This is expected because stronger convection transports the inlet temperature distribution downstream and limits the relative influence of thermal diffusion and viscous heating.

Reducing the convection coefficient weakens this transport. Consequently, thermal diffusion and viscous dissipation have a greater influence on the temperature field. The resulting distribution departs further from the initial linear profile, with a more pronounced temperature increase in the upper region of the channel.

The three cases are compared in Figure 8. The differences illustrate how the relative strengths of convection, thermal diffusion, and viscous heating determine the resulting temperature distribution.

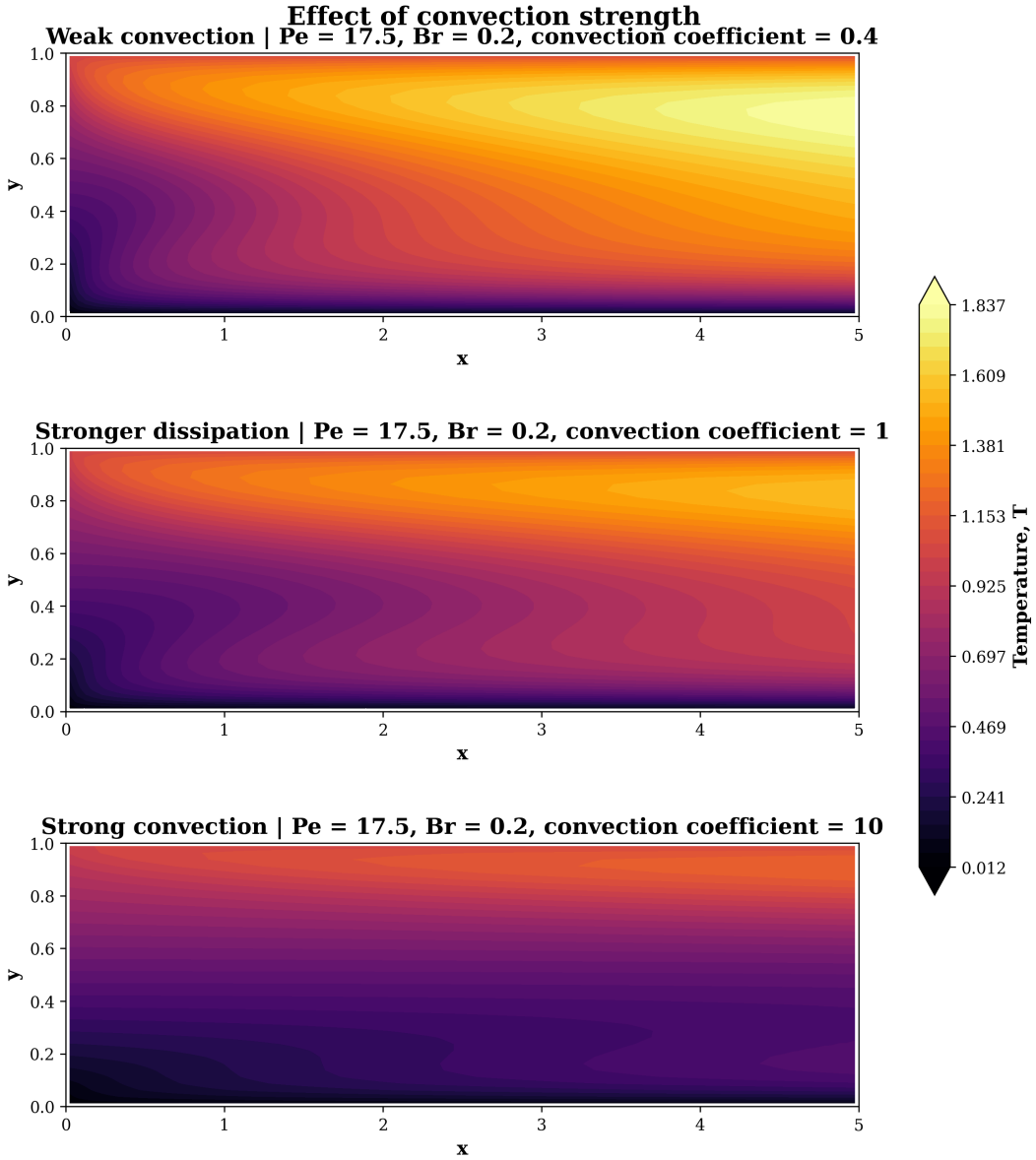


Figure 8: Effect of the convection coefficient on the steady-state temperature distribution for $Pe = 17.5$ and $Br = 0.2$.

7.6.3 Effect of the Peclet Number

The Peclet number represents the relative strength of convective transport to thermal diffusion. Its effect was studied at $Pe = 1$, 17.5, and 50. In these cases, the ratio Br/Pe was held constant at 0.004, so the coefficient of the viscous dissipation source remained unchanged.

A low Peclet number corresponds to stronger thermal diffusion relative to convection. Since the initial and inlet temperature profiles are identical, the temperature field tends to remain close to the linear distribution. Thermal diffusion also smooths temperature variations produced by the viscous heating source.

As the Peclet number increases, the relative influence of thermal diffusion decreases. The temperature field therefore exhibits a greater departure from the initial profile under the combined effects of convection and viscous dissipation. In particular, the temperature variations generated by the dissipation source are less strongly smoothed by diffusion.

Figure 9 compares the temperature fields at the three Peclet numbers. The results demonstrate the influence of thermal diffusion on the balance between convective transport and viscous heating.

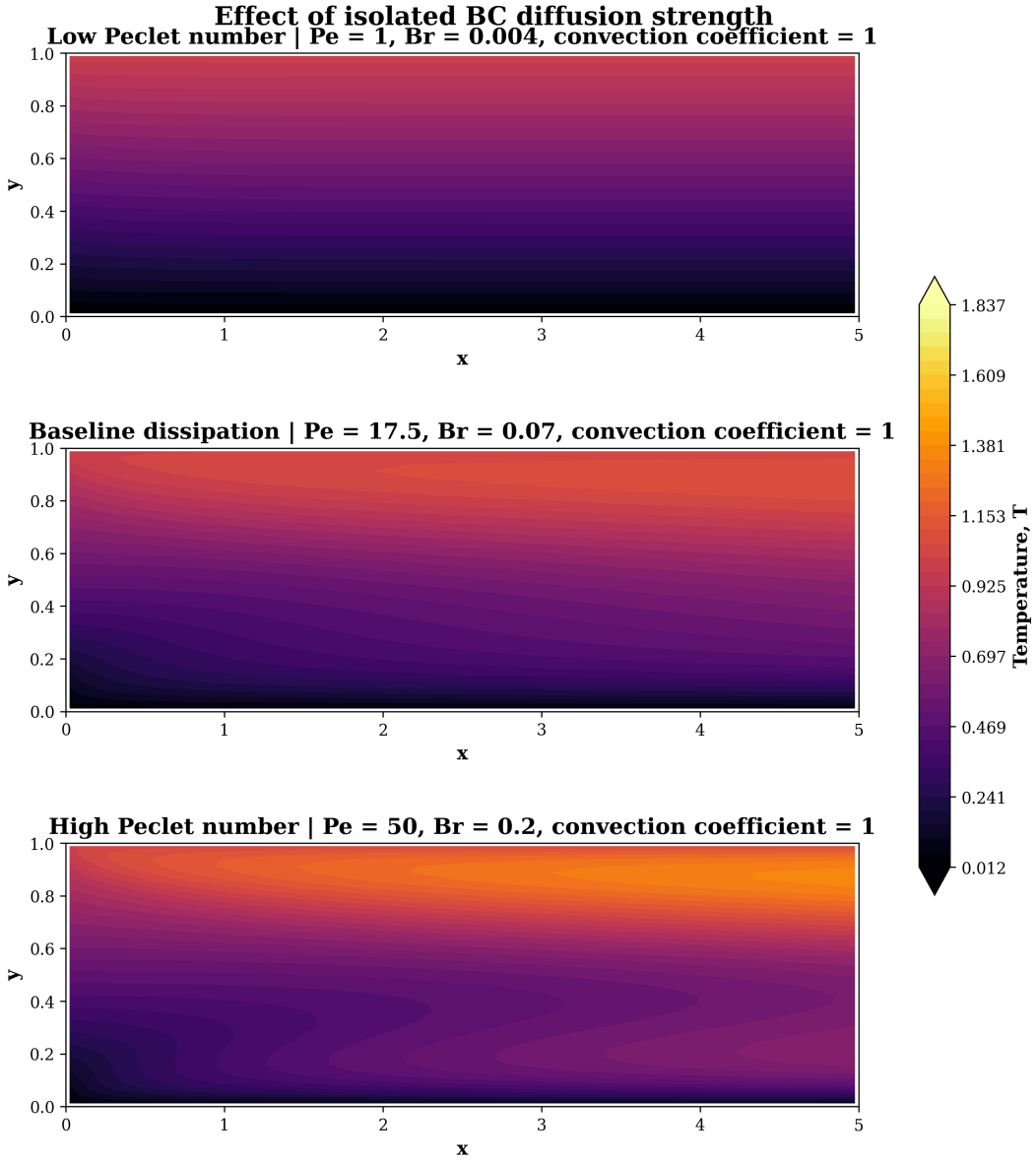


Figure 9: Effect of the Peclet number on the steady-state temperature distribution, with $Br/Pe = 0.004$ and $\gamma_c = 1$.

7.6.4 Temperature Cross-Sections

To examine the combined effects of convection and viscous dissipation in more detail, temperature profiles were extracted at two streamwise locations, $x/L_x = 0.15$ and 0.85. Three cases were selected for this comparison:

- Case 1: $Pe = 17.5$, $Br = 0.07$, and $\gamma_c = 1$.
- Case 2: $Pe = 17.5$, $Br = 0.2$, and $\gamma_c = 1$.
- Case 3: $Pe = 17.5$, $Br = 0.2$, and $\gamma_c = 0.4$.

The first two cases have identical Peclet numbers and convection coefficients. Their comparison therefore isolates the effect of viscous dissipation. Increasing the Brinkman number from 0.07 to 0.2 produces a more pronounced curvature in the wall-normal temperature profiles. This reflects the stronger contribution of viscous heating to the energy balance.

The second and third cases have the same Peclet and Brinkman numbers, but different convection coefficients. Reducing γ_c from 1 to 0.4 weakens convective transport and increases the relative influence of thermal diffusion and viscous heating. The resulting profiles exhibit a warmer upper region and a less pronounced wave-like variation, consistent with the smoothing effect of thermal diffusion.

Together, these comparisons show that the temperature profiles result from the combined effects of convective transport, thermal diffusion, and viscous dissipation. Their relative contributions determine both the temperature level and the shape of the wall-normal profiles.

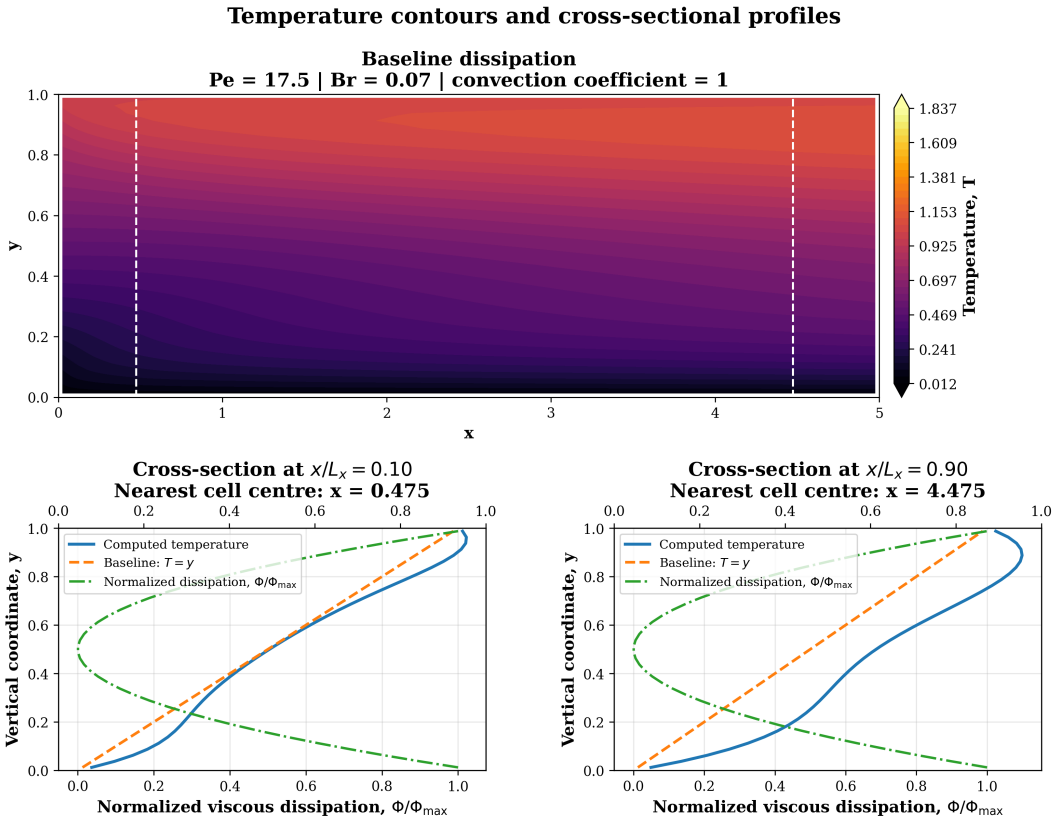


Figure 10: Temperature contours and wall-normal temperature profiles for $Pe = 17.5$, $Br = 0.07$, and $\gamma_c = 1$.

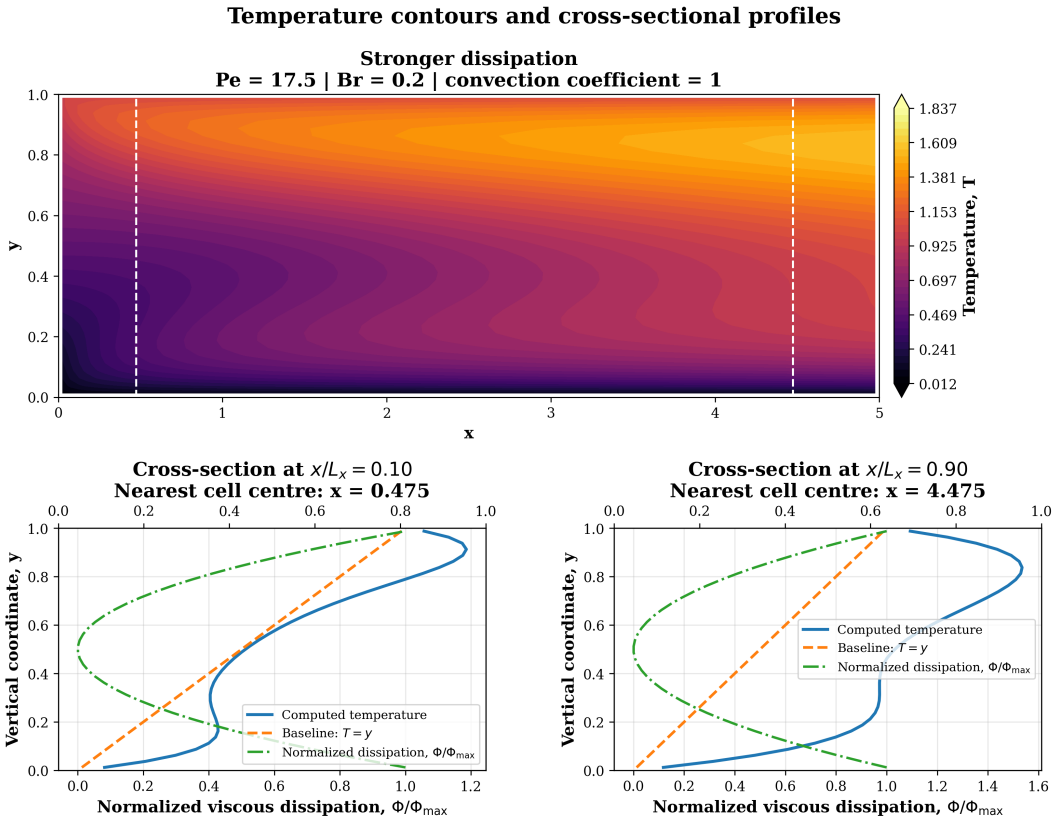


Figure 11: Temperature contours and wall-normal temperature profiles for $Pe = 17.5$, $Br = 0.2$, and $\gamma_c = 1$.

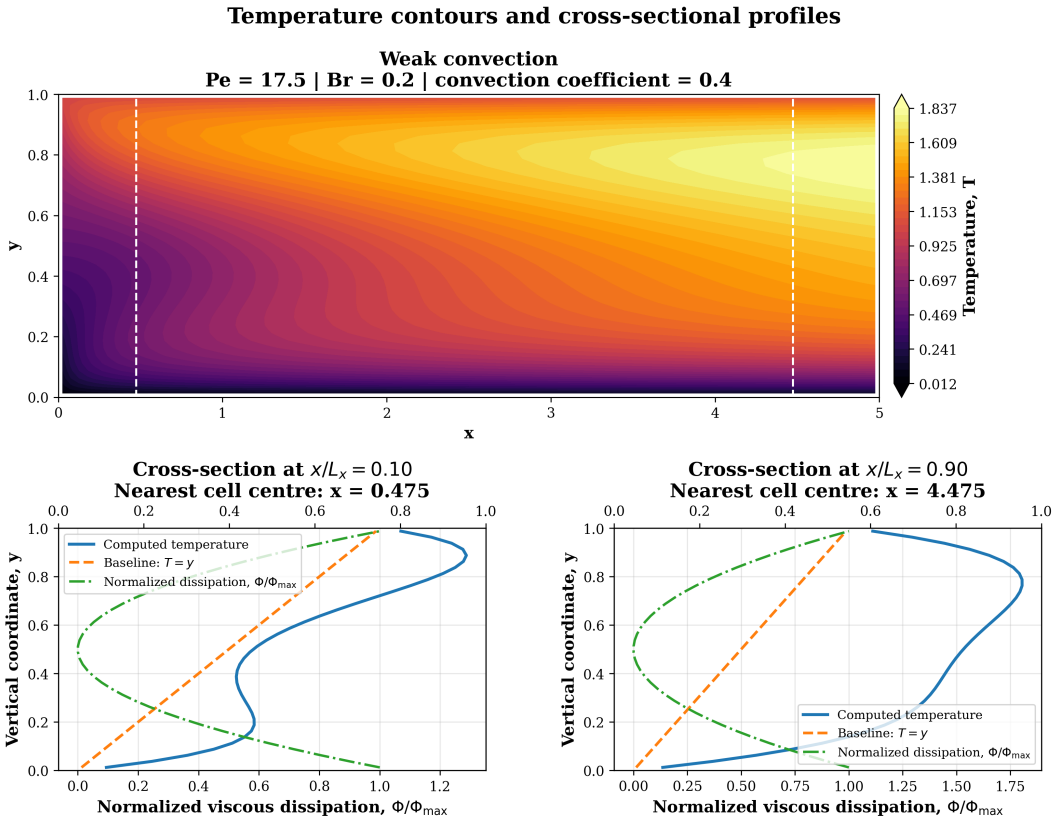


Figure 12: Temperature contours and wall-normal temperature profiles for $Pe = 17.5$, $Br = 0.2$, and $\gamma_c = 0.4$.

8. Conclusion

A finite-volume solver was developed to investigate the steady-state temperature distribution in a two-dimensional channel flow, accounting for convection, thermal diffusion, and viscous dissipation. The governing energy equation was discretized in space using central-difference approximations, resulting in a system of ordinary differential equations. Several time-integration methods were implemented to advance this system toward steady state, including the approximate factorization method (AFI), a sparse direct implicit method, and the explicit second- and fourth-order Runge–Kutta methods.

The numerical stability analysis showed that the explicit Runge–Kutta methods require sufficiently small time steps to maintain stability. The eigenvalue analysis provided estimates of their stability limits, which were then used to select suitable time steps for the solver comparison. The implicit methods allowed larger time steps, although their computational cost depended on the solution procedure and implementation.

The performance comparison showed that all four methods reached the prescribed residual tolerance of 10^{-12} on the 50×20 grid. Among the tested methods, the sparse direct solver achieved the shortest computation time, with a median runtime of approximately 0.14 s. The AFI, RK2, and RK4 methods required approximately 6.28, 8.01, and 10.72 s, respectively. Although RK4 required fewer iterations than RK2, its additional stages per iteration resulted in a higher total computation time. These results apply to the selected grid, time steps, convergence criterion, and implementations; they do not establish a general performance ranking for other

configurations.

The temperature-field evolution illustrated the interaction between convection, thermal diffusion, and viscous dissipation during the approach to steady state. The parameter study further clarified their individual contributions. Increasing the Brinkman number strengthened viscous heating and produced larger departures from the initial linear temperature profile. Increasing the convection coefficient enhanced the downstream transport of the inlet temperature distribution, whereas reducing it allowed thermal diffusion and viscous dissipation to have a greater influence. In the Peclet-number study, the ratio Br/Pe was kept constant so that the viscous-dissipation coefficient remained unchanged while the thermal-diffusion coefficient decreased with increasing Peclet number. The resulting temperature distributions demonstrated how the balance between these mechanisms determines the shape and magnitude of the temperature field.

Overall, the project provided a numerical framework for solving the two-dimensional energy equation and examining the effects of its governing parameters. The results also demonstrated that solver performance depends on the time-integration method, time-step selection, and computational implementation. Further work could include a more detailed grid-convergence study and comparisons of computational cost and accuracy over multiple grid resolutions.